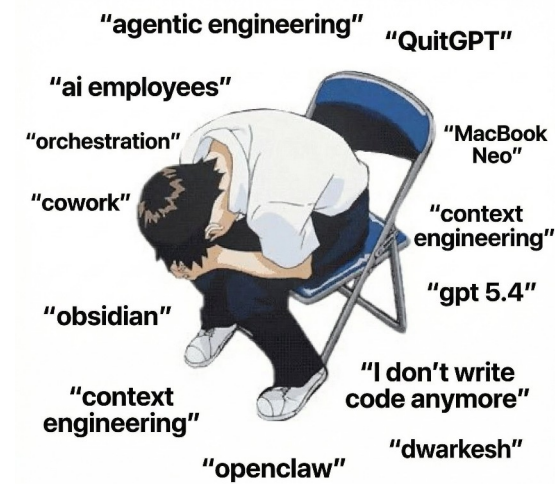
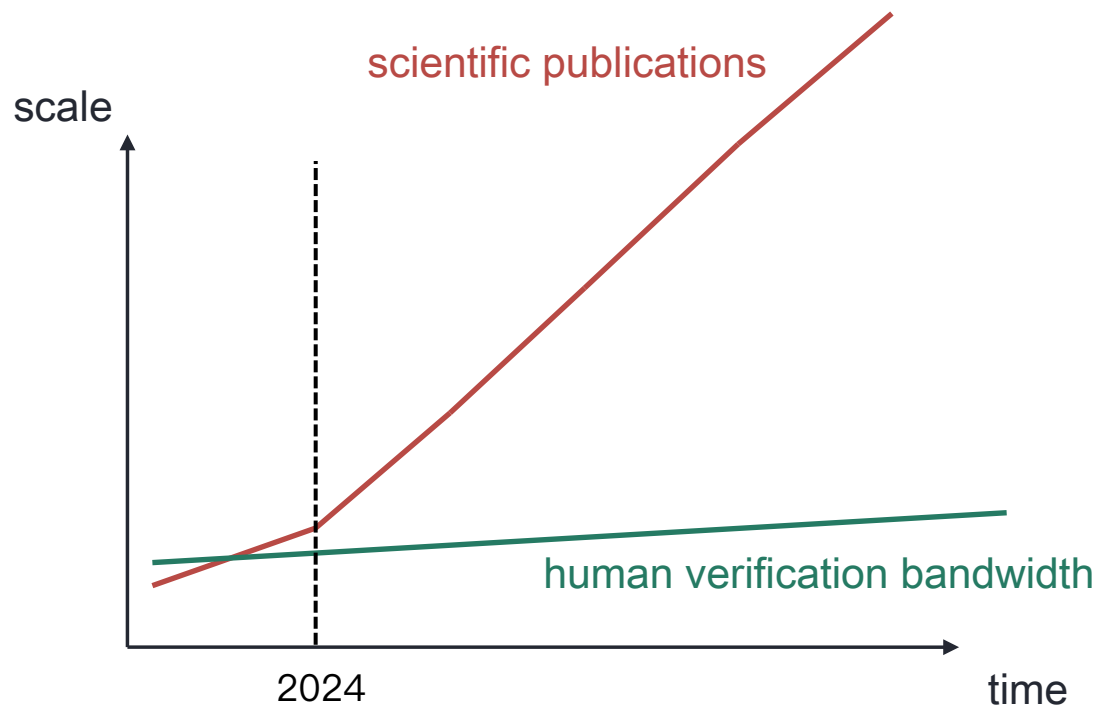


Some thoughts about LLM for theoretical research

Yingjian Liu

The AI crisis in theoretic research

(AI) read $\neq$ checked $\neq$ reusable

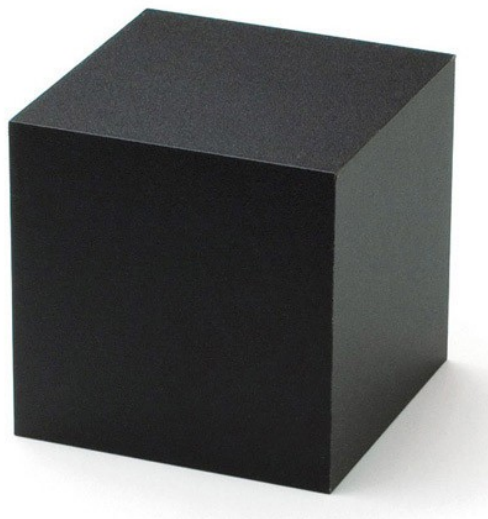


- Human researcher will be drowned in infinite "recent progress" papers.
- Any open problem be solved increases the confidence of AGI, and also the anxiety of **our career**.
- Later work still needs the exact imported claim and **verification**.
- Claims and semantic content are often exaggerated.

Before asking whether we can trust what it writes,
look at what it is doing when it writes.

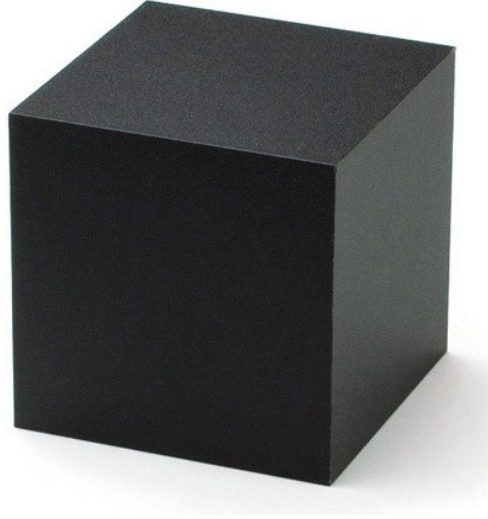
Nothing in the next four frames is a criticism. It is a description.

What is inside the black box of LLM reasoning



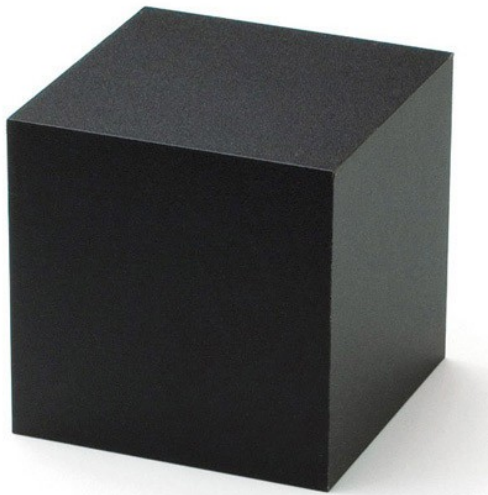
$f : \text{String} \rightarrow \text{String}$

What is inside the black box of LLM reasoning

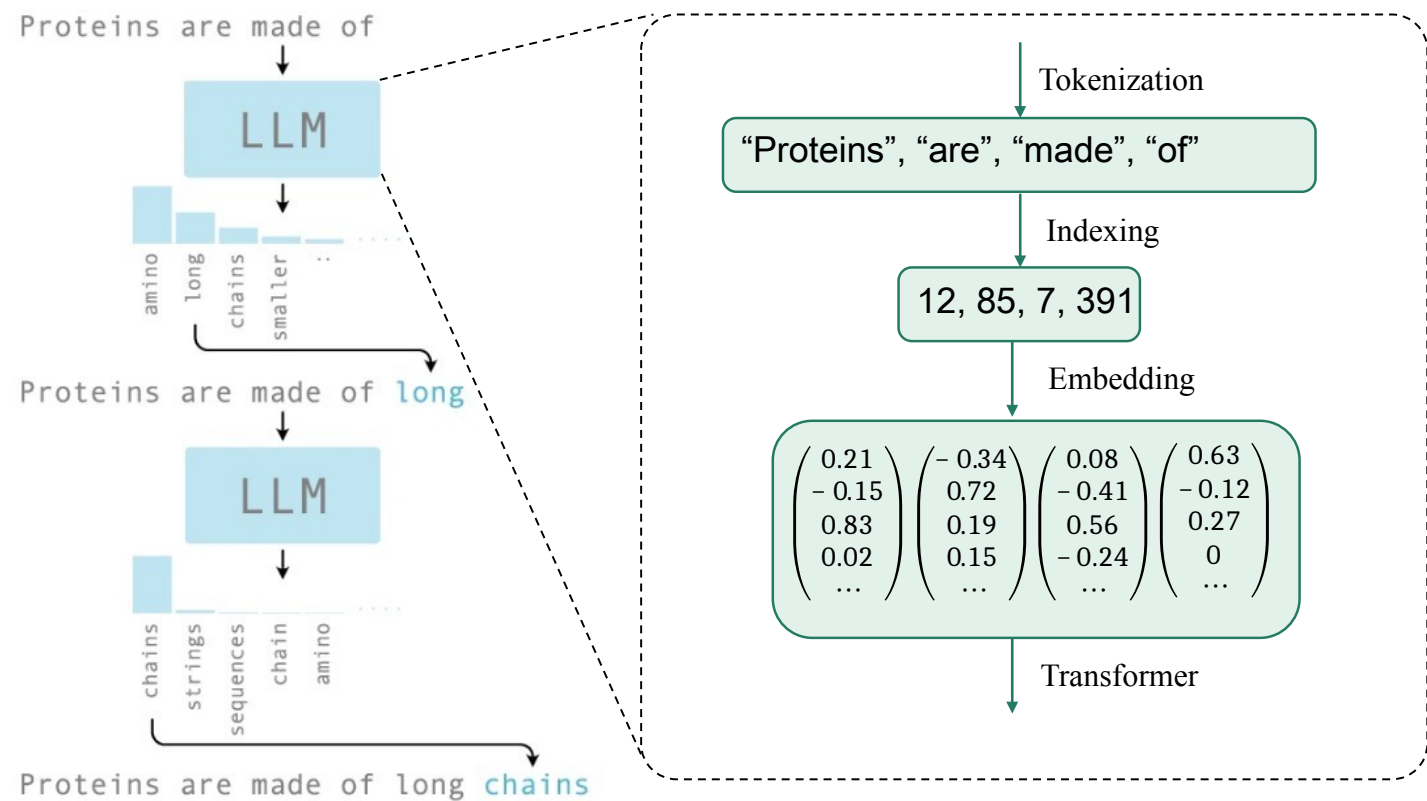


f : String $\rightarrow$ token $\rightarrow$ prob. distribution $\rightarrow$ (next) token $\rightarrow$ String

What is inside the black box of LLM reasoning



$f : \text{String} \rightarrow \text{token} \rightarrow \text{prob. distribution} \rightarrow (\text{next}) \text{ token} \rightarrow \text{String}$



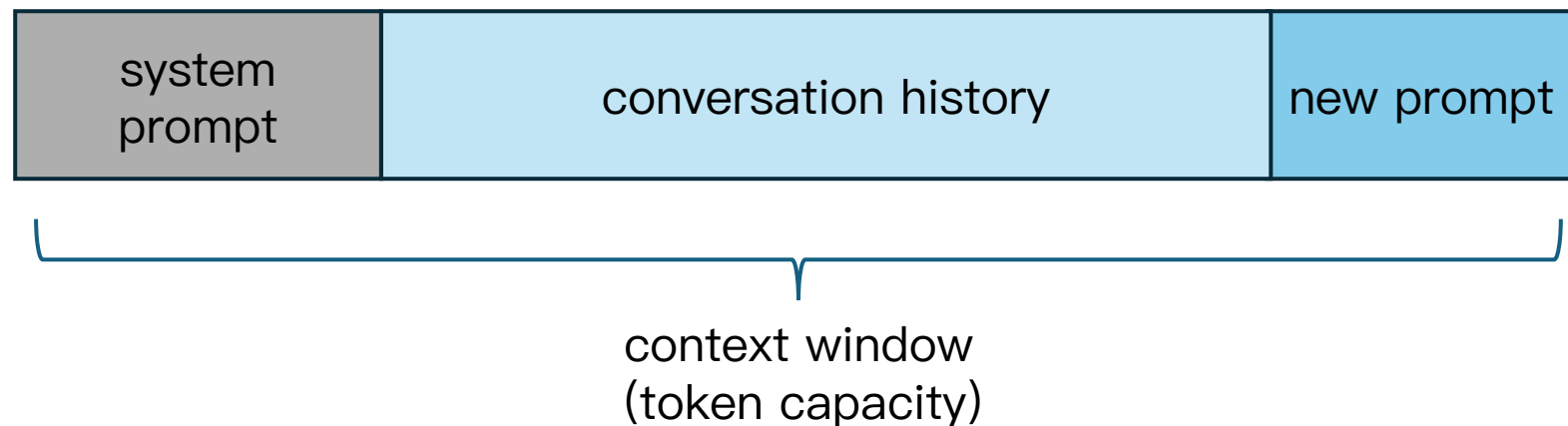
LLM outputs a conditioned probabilistic distribution $\mathbf{P}(\text{next token}|\text{context})$, then **samples**.

Model Momery

- Normally, the conversation with LLM is a forward pass and **does not update any model weights**
- Model Momery: saves previous prompts and answers **re-sends** the history every time

```
Turn 1  
[User: I am a physicist.]  
→ LLM → [Response: Understood.]
```

```
Turn 2  
[User: I am a physicist.]  
[Response: Understood.]  
[User: Explain entropy at my  
level.]  
→ LLM → [Response: ...]
```

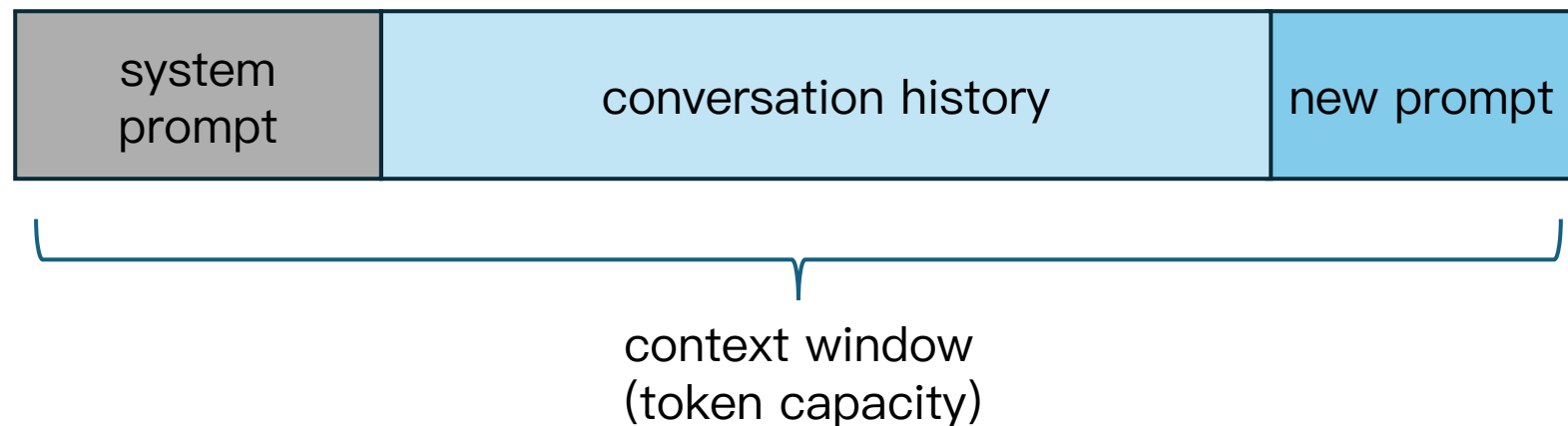


Model Momery

- Normally, the conversation with LLM is a forward pass and **does not update any model weights**
- Model Momery: saves previous prompts and answers **re-sends** the history every time

```
Turn 1
[User: I am a physicist.]
→ LLM → [Response: Understood.]
```

```
Turn 2
[User: I am a physicist.]
[Response: Understood.]
[User: Explain entropy at my
level.]
→ LLM → [Response: ...]
```



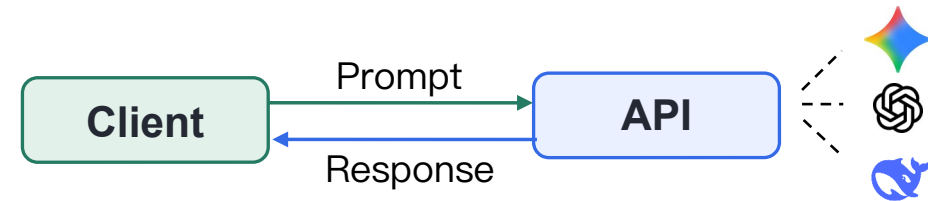
When the context window is exceeded, the compression/truncation/selection must be applied

API/Agent/MCP/schema/skills ... prompt sorcery from LLM era

Numpy API

```
import numpy as np  
result = np.mean([1, 2, 3])
```

LLM API



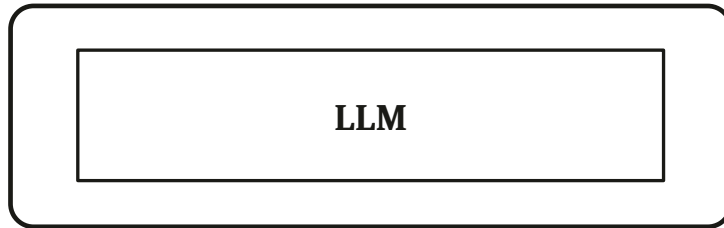
- Agent is a **system** with a **LLM as its core** to execute high-level tasks: LLM decides next step



LLM

text → a distribution over next tokens

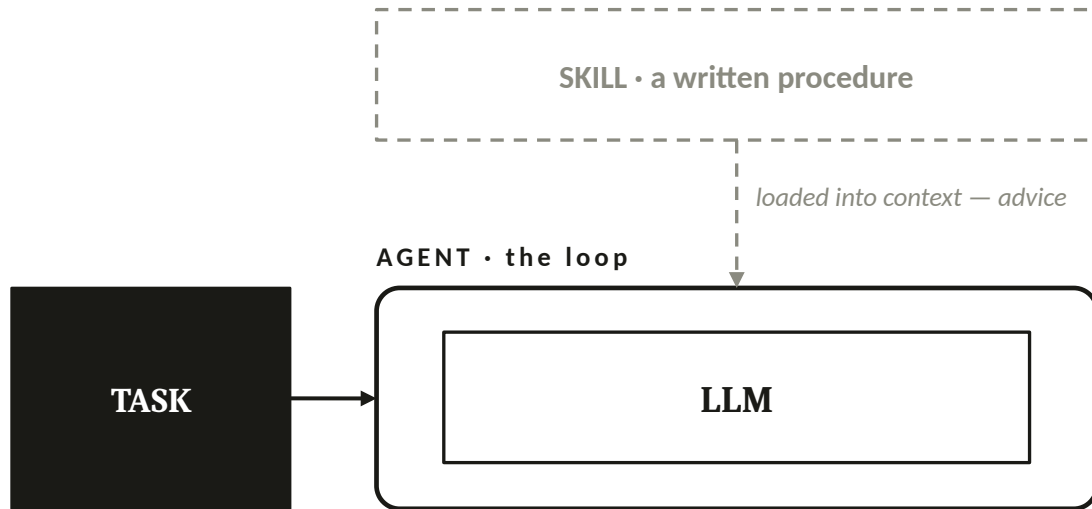
AGENT • the loop



The agent calls the model repeatedly, decides what to do next, and stops when it judges the work done.

A skill is a structural prompt document

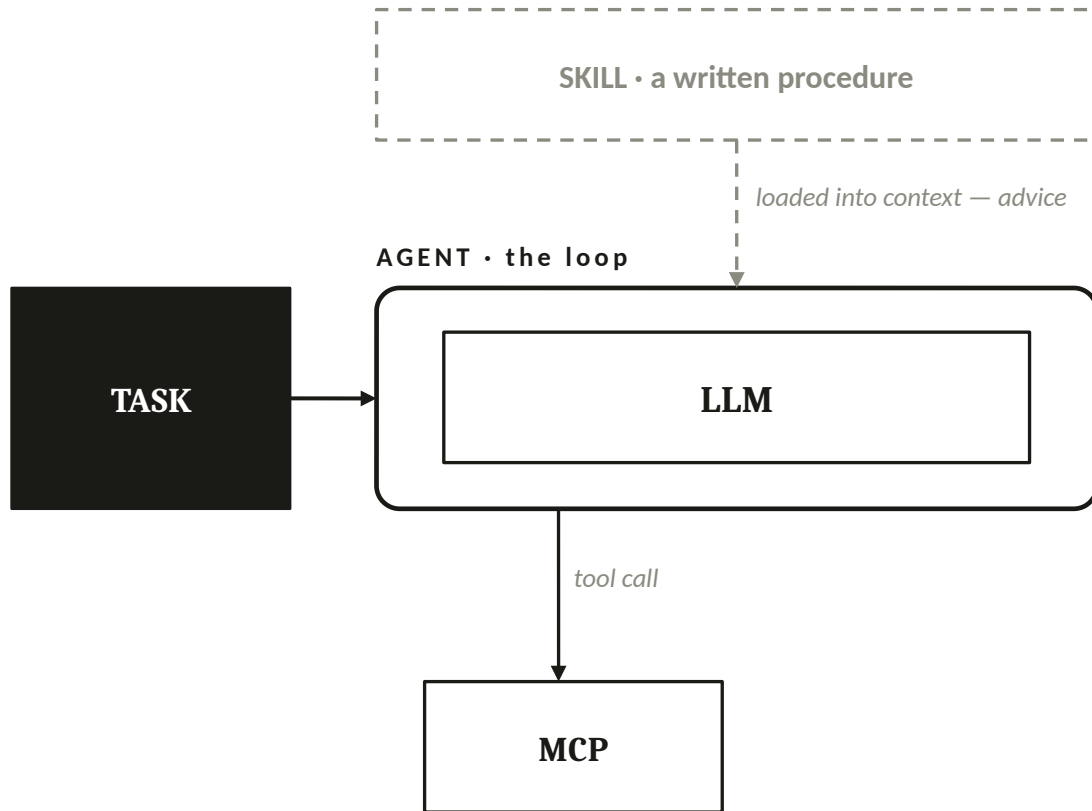
4 / 9



A protocol loaded into the context when the task matches it — drawn dashed, because it is advice. The model can read it and not follow it, and nothing records that it did.

MCP: Model Context Protocol

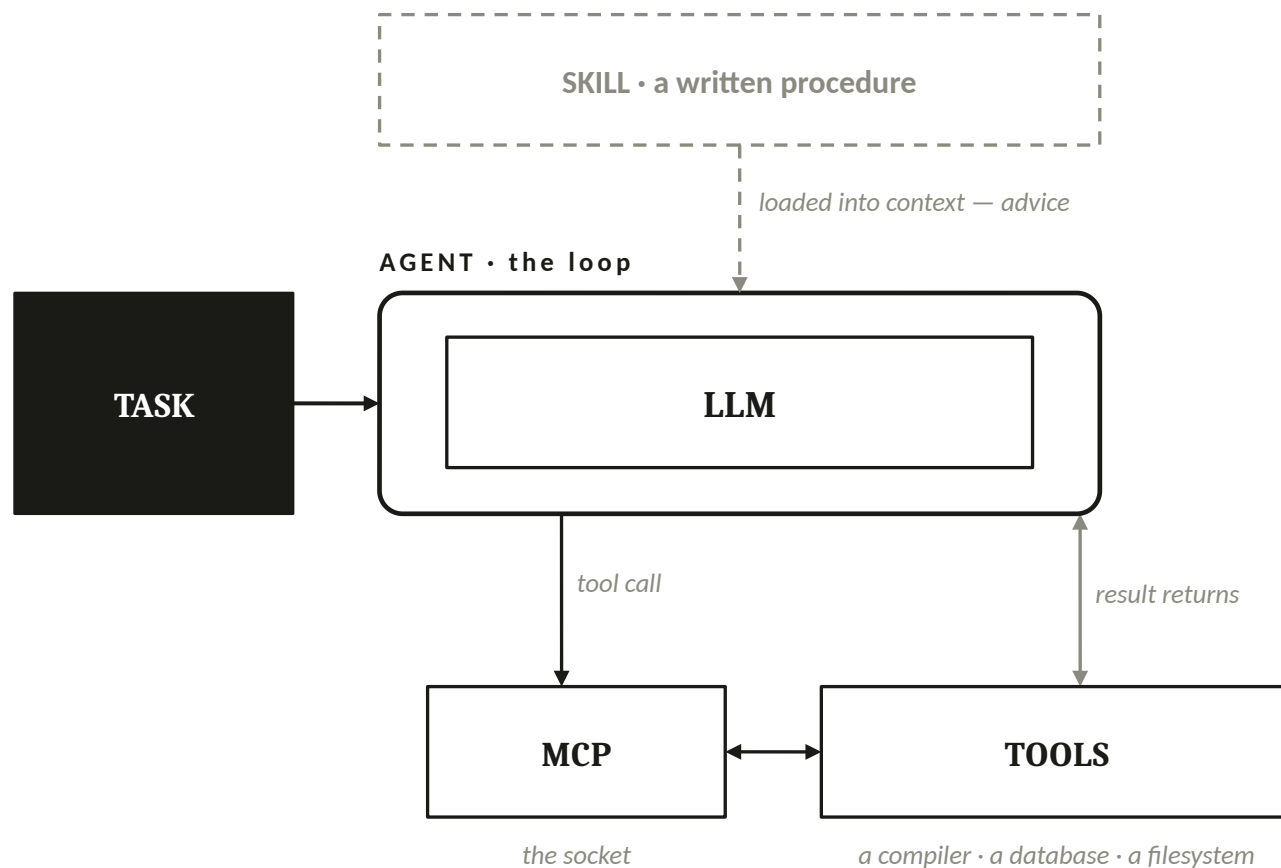
5 / 9



A published convention for exposing a tool so that any model can call it.
It standardises the plug.

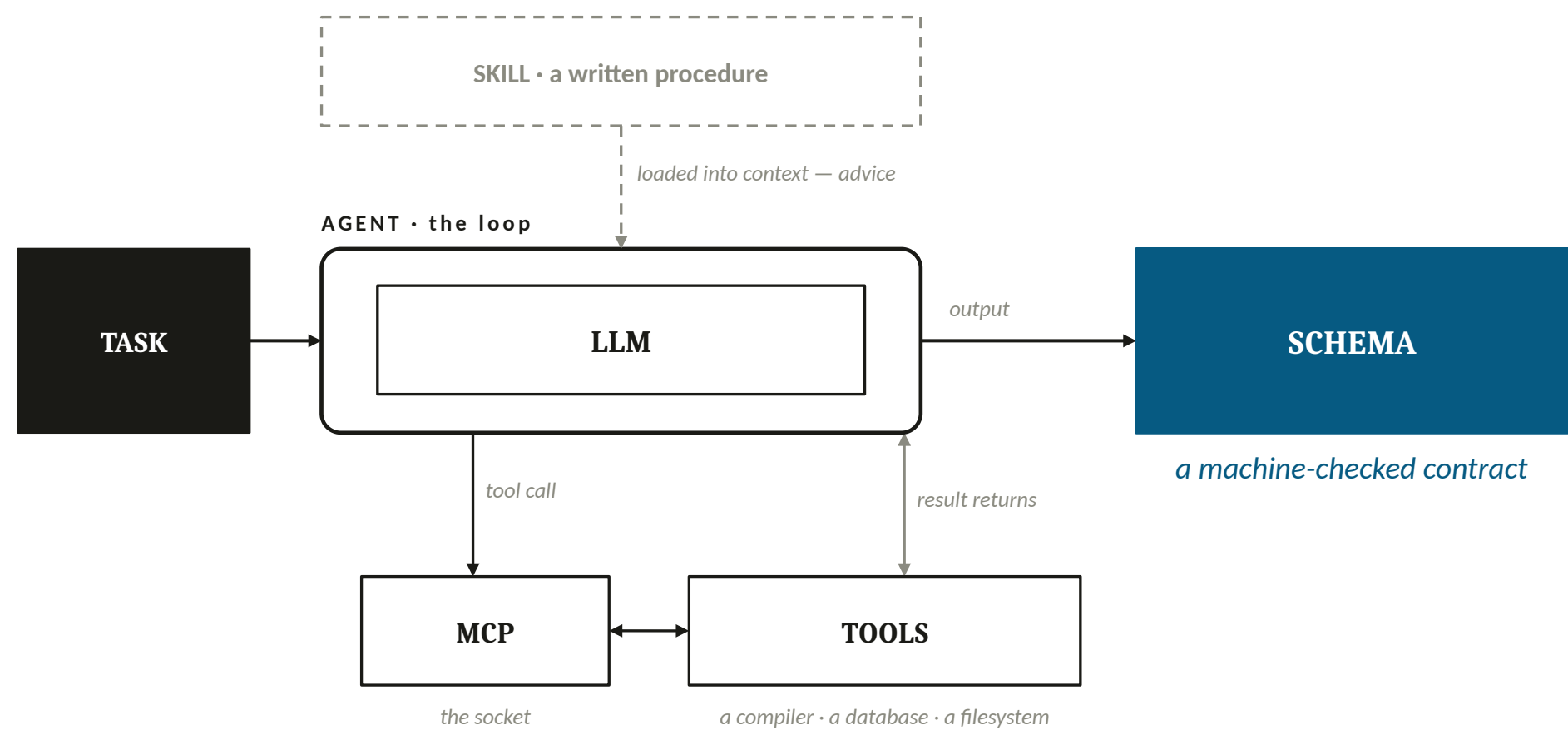
Agent uses tools and the loop

6 / 9



A compiler, a database, a filesystem. The result returns to the loop, which calls the model again.

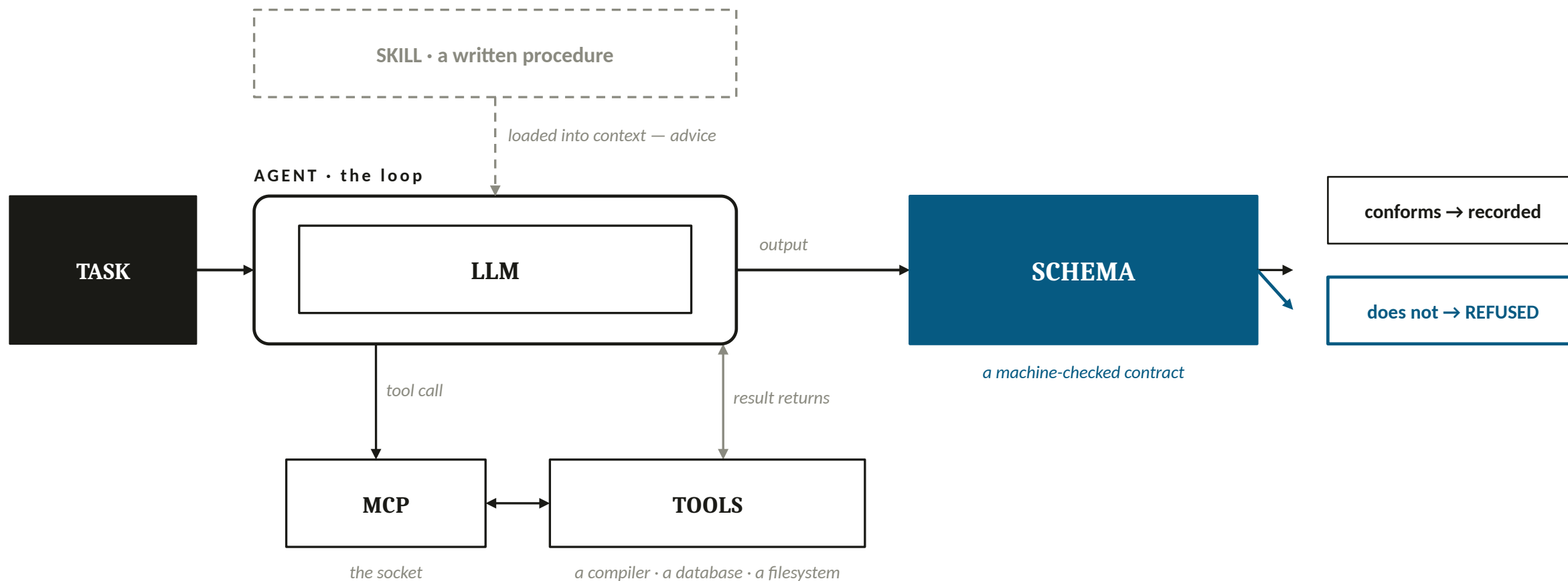
Schema: a machine-checked contract



A machine-checked contract: A valid record must output correct types with respect to contract

Schema: a machine-checked contract

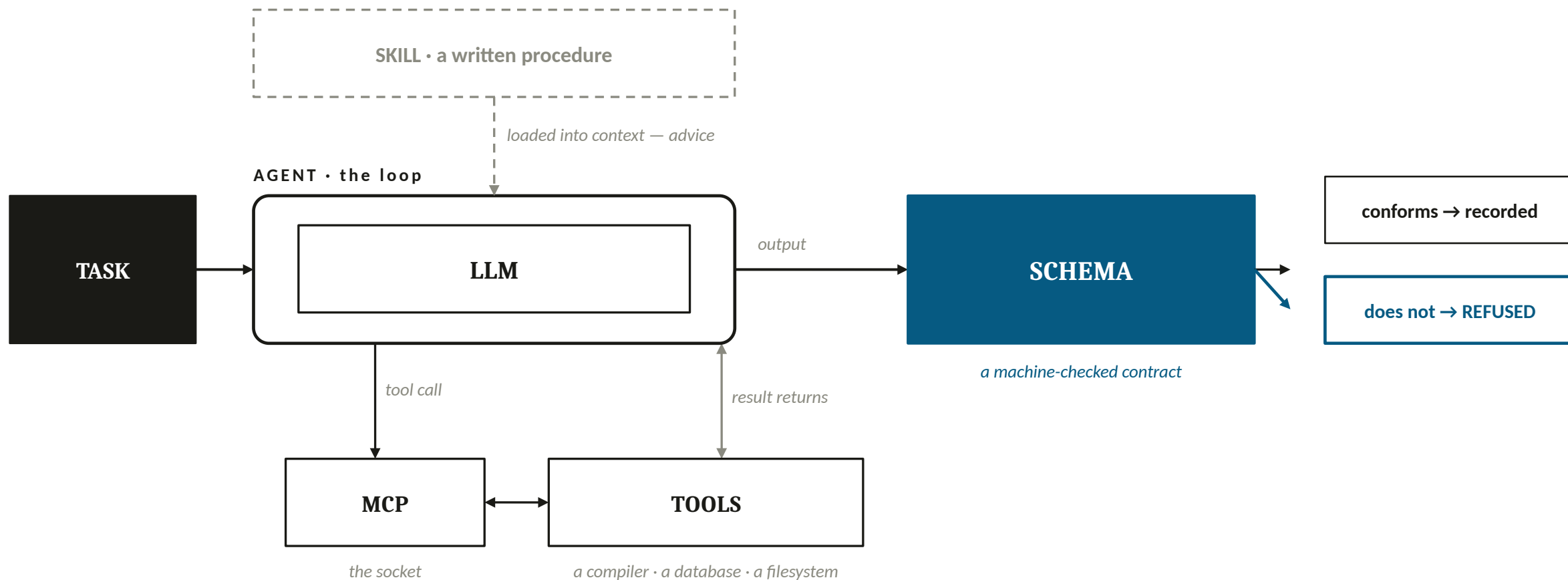
8 / 9



A record that does not conform is not repaired or accepted with a warning, but it is refused.

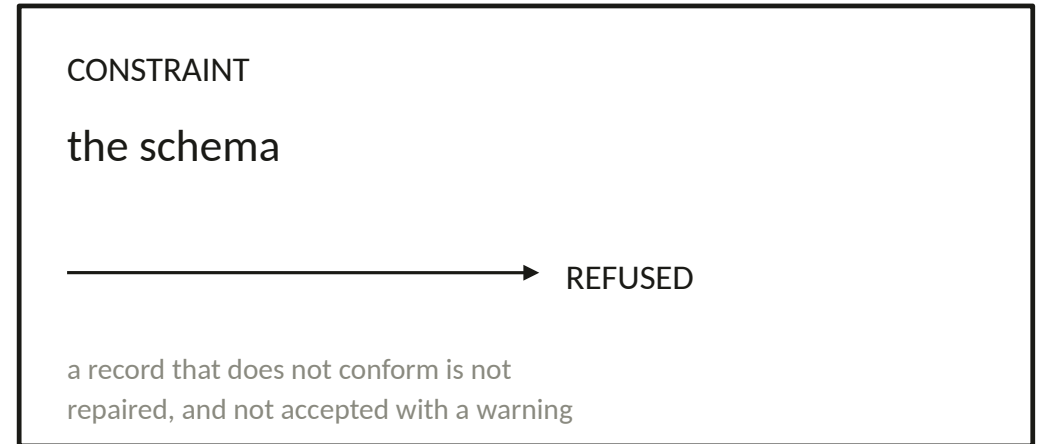
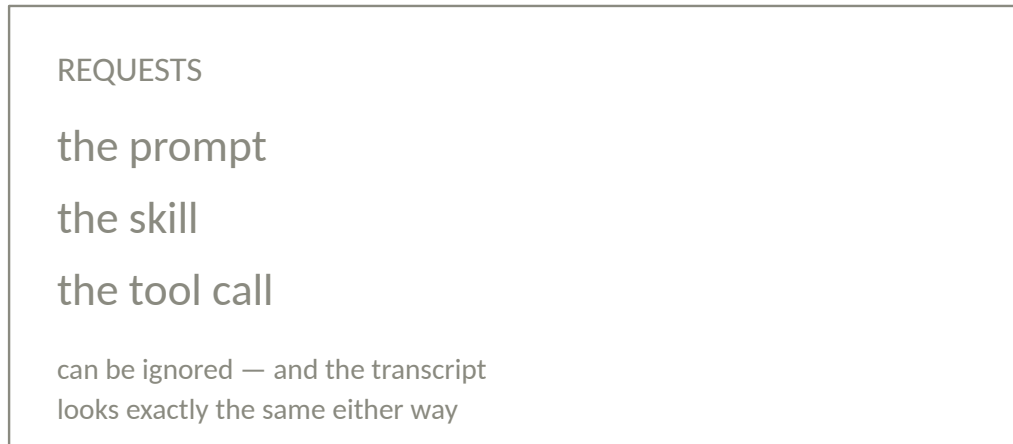
Schema: a machine-checked contract

9 / 9



The prompt, the skill and the tool call are all requests — they can be ignored, and the transcript looks the same either way. Only the schema can make an answer impossible to express.

A request can be ignored. A constraint cannot.



A paper's claims are requests too.

The rest of this talk is an attempt to make them refusable:
one paper, broken into claims that each carry what they rest on.

“Please help me ...”

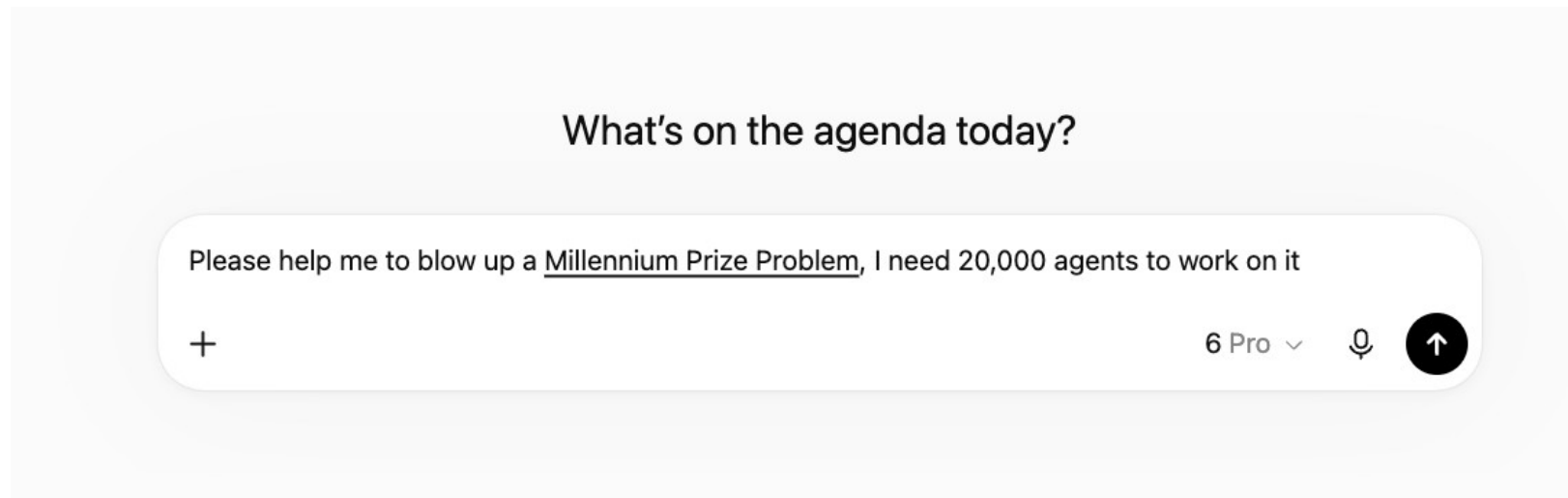
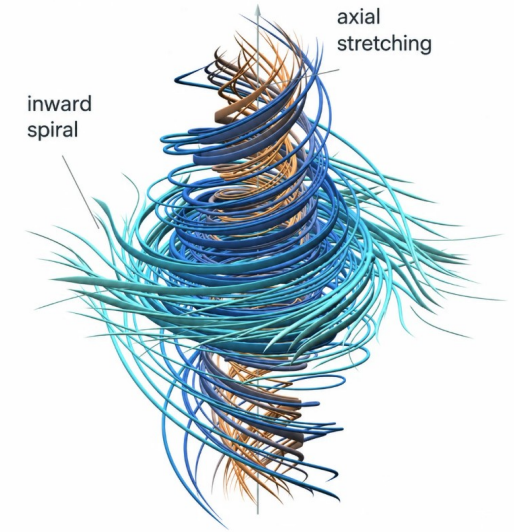


Learn backwards

“Please help me ...”



Learn backwards

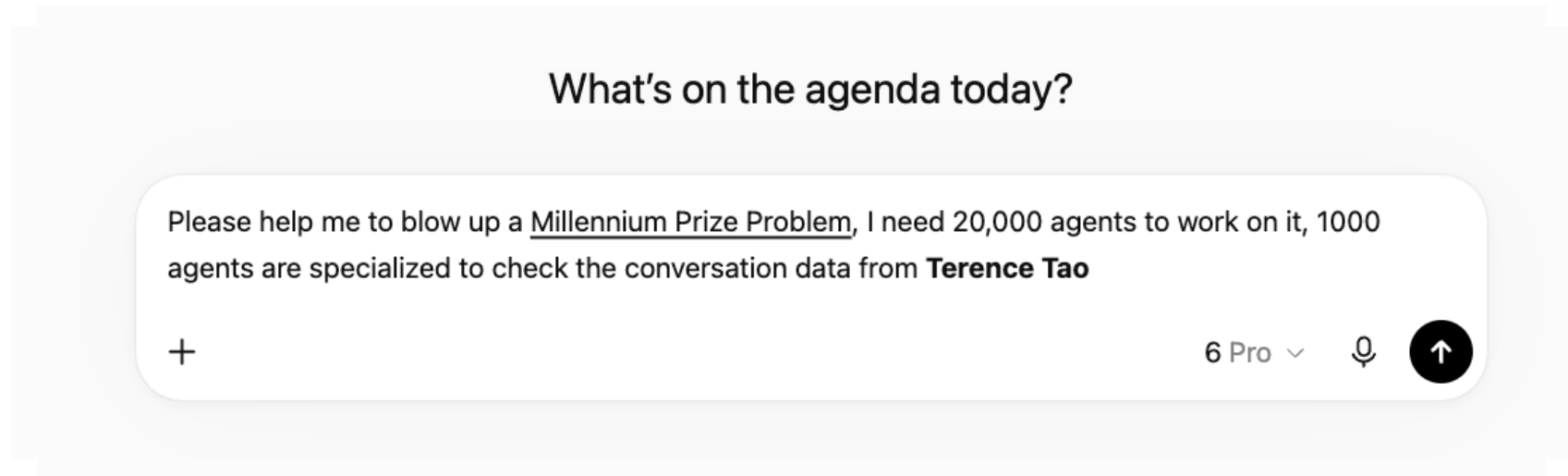
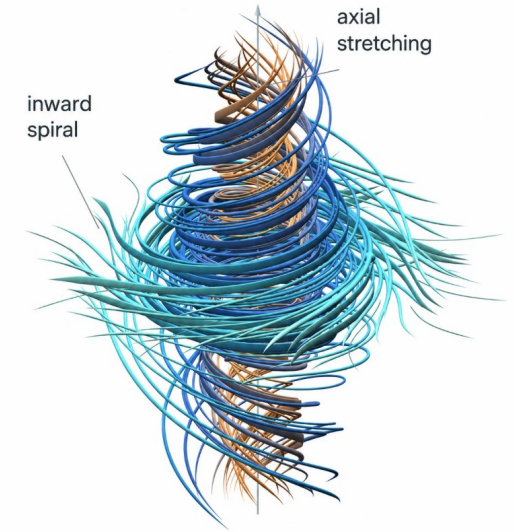


Reasoning forwards

“Please help me ...”



Learn backwards



Reasoning forwards

Or, sometimes together ...

What we actually care, and ... what we can trust? (as theorists)

The story

What did the paper actually state?

Mathematical correctness

Does the conclusion follow from encoded assumptions?

Computational reproducibility

Can the numerical result be reproduced?

Physical-semantic alignment

Does the formal object represent the intended physics?

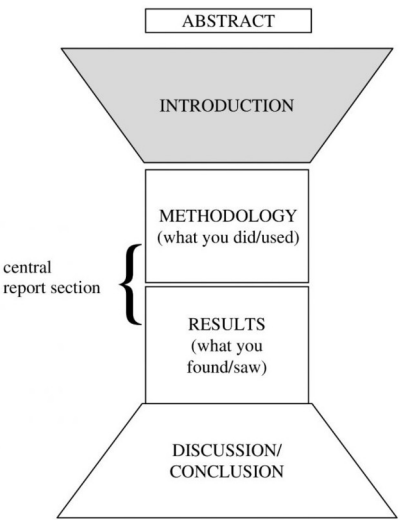
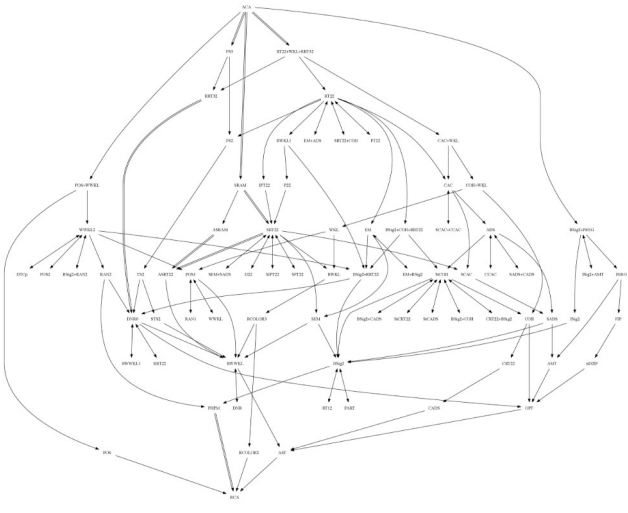
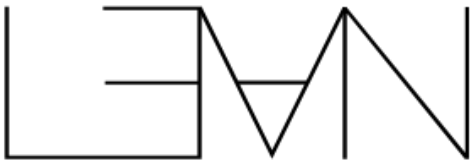


Fig. 1. The shape of a research article or thesis.

Paper formatting, SUNY



The reverse mathematics zoo,
founded by Damir Dzhafarov , 2019



inductive BoolExpr **where**
| var (name : String)
| val (b : Bool)
| or (p q : BoolExpr)
| not (p : BoolExpr)

The Lean 4 Theorem Prover and Programming Language
by Moura L, Ullrich S. 2021



The story

What did the paper actually state?

Mathematical correctness

Does the conclusion follow from encoded assumptions?

Computational reproducibility

Can the numerical result be reproduced?

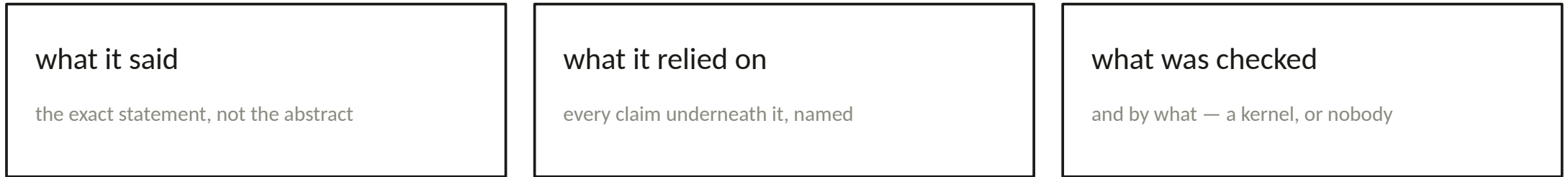
Physical-semantic alignment

Does the formal object represent the intended physics?

one paper · one theorem · and what it actually rests on

not whether the formalisation means the physics you had in mind — no kernel judges that
and not reproducibility — there is no numerical claim in the case study

For later work to use a conclusion, it has to know three things



One real paper, one real run. Every number after this frame is read out of the logs, not estimated.

Where nobody measured it, the frame says so.

Imaging paper as an agent

[nature](#) > [articles](#) > [article](#)

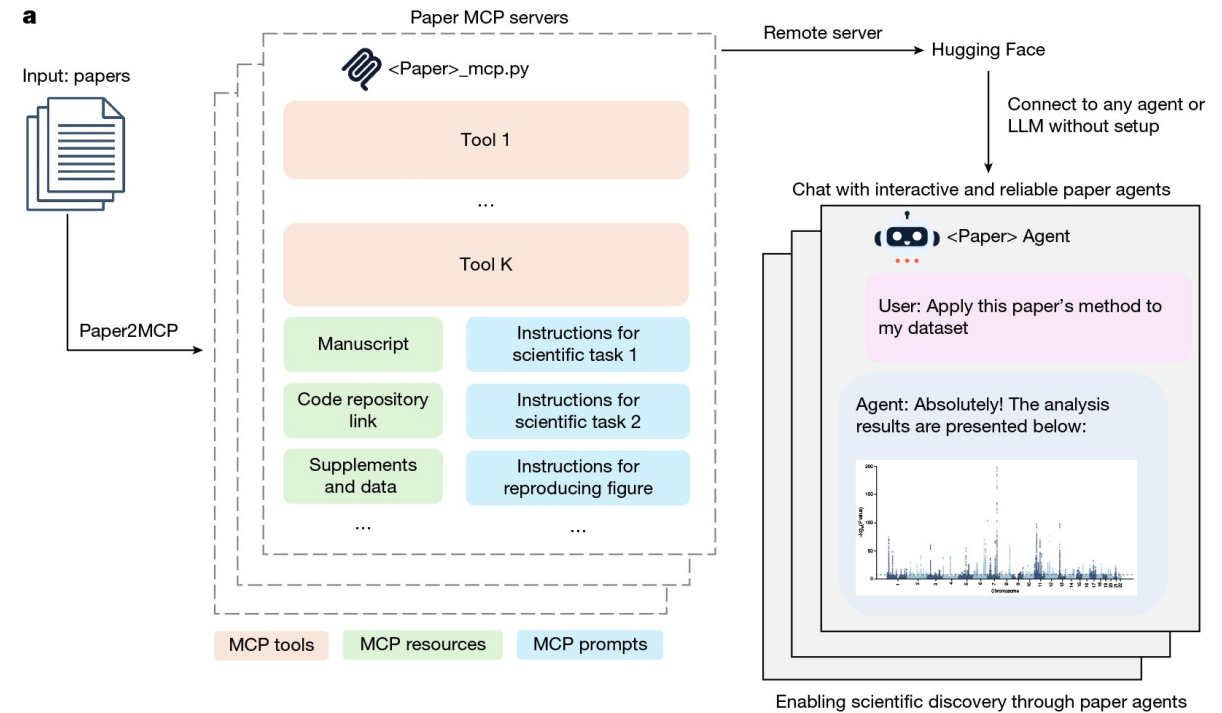
Article | [Open access](#) | Published: 16 September 2026

Reimagining research papers as interactive and reliable AI agents

[Jiacheng Miao](#) ✉, [Joe R. Davis](#), [Yaohui Zhang](#), [Jonathan K. Pritchard](#) & [James Zou](#) ✉

[Nature](#) (2026) | [Cite this article](#)

[Save article](#)



Agentic Publication Protocol: An Attempt to Modernize Scientific Publication

Sirui Lu^{1,2} and Xiao-Liang Qi^{3,*}

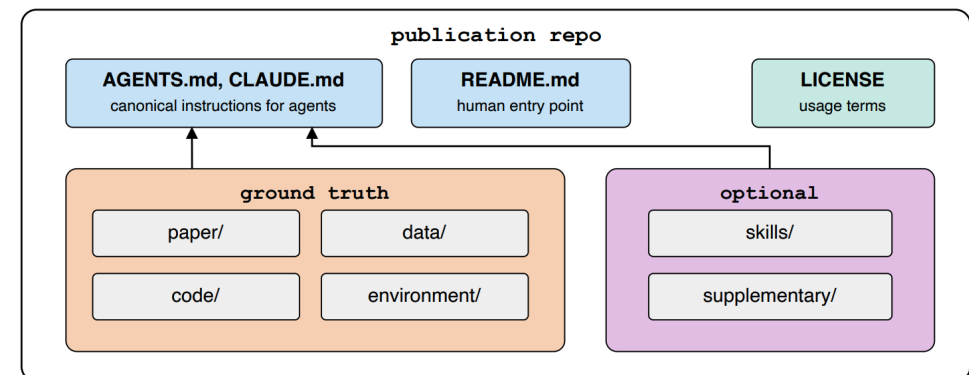
¹Max-Planck-Institut für Quantenoptik, Garching, Germany

²Munich Center for Quantum Science and Technology (MCQST),
Schellingstrasse 4, D-80799 München, Germany

³Leinweber Institute for Theoretical Physics, Stanford University, Stanford, CA, USA

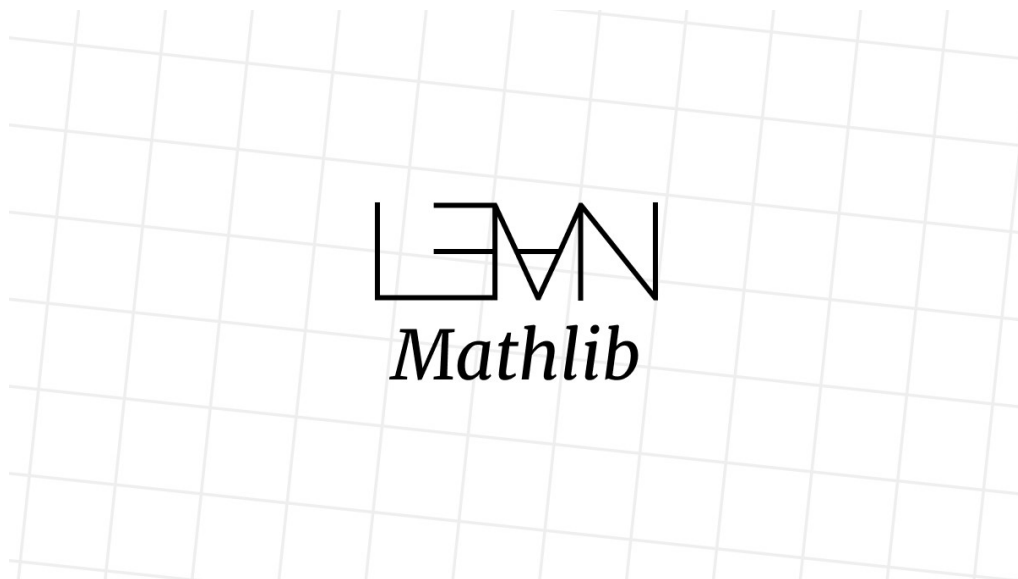
*xlqi@stanford.edu

Canonical structure of an APP publication repo



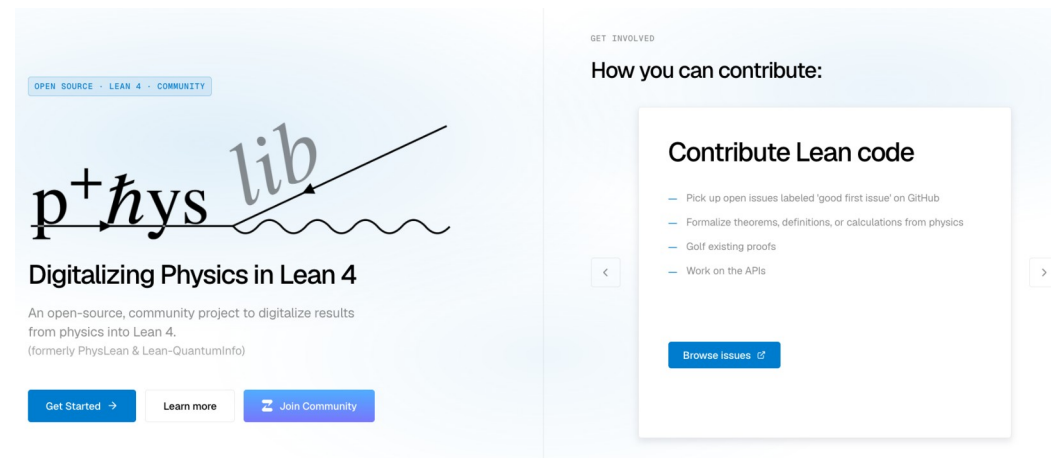
A paper no need to be static, but an agent

How formalization works



Mathlib — the mathematics

groups, measure, linear algebra, polytopes



physlib — the physics

an open-source community project, still being built

Lean 4 — how to read it

Six lines. On the left is what you type; on the right is what each line says.

```
def double (n : Nat) : Nat :=  
  2 * n
```

```
theorem double_eq_add (n : Nat) :  
  double n = n + n := by
```

```
  unfold double  
  omega
```

DEFINE a thing called double. Give it a whole number n ...

... and what comes back is 2 times n . Nothing is claimed yet.

CLAIM, and give the claim a name. For every whole number n ...

... double n is the same as $n + n$. "by" means: proof follows.

step 1 — replace double by what it stands for: $2 * n = n + n$.

step 2 — omega settles arithmetic statements like that one.

Nothing here is prose. That line IS the theorem — not a description of one.

Which is the whole trade: you give up the words, and in exchange the claim becomes something a machine can check, and refuse.

It compiles, or it does not. There is no third answer.

accepted

each step re-checked by a kernel of a few thousand lines
that does not know who wrote it, or how tired it is

rejected

and it names the step it would not take —
which is a research to-do list, not a failure

There is exactly one way to cheat, and it is a keyword:

sorry

— accept this claim without a proof. It compiles. It is also recorded, forever, in the file.

Anybody can write it, and it compiles. The difference from prose is that it is written down
— it sits in the file, and anybody who looks can find every one of them.

Nobody proves anything from nothing

Every proof stands on definitions somebody else already wrote and checked.

9,146

.lean files

Mathlib

the mathematics. Groups, measure, linear algebra, polytopes. Begun 2017.

927

.lean files

physlib

the physics. An open-source community project to digitalize results from physics into Lean 4. Begun 2024.

Ten times the size, and seven years of a head start.

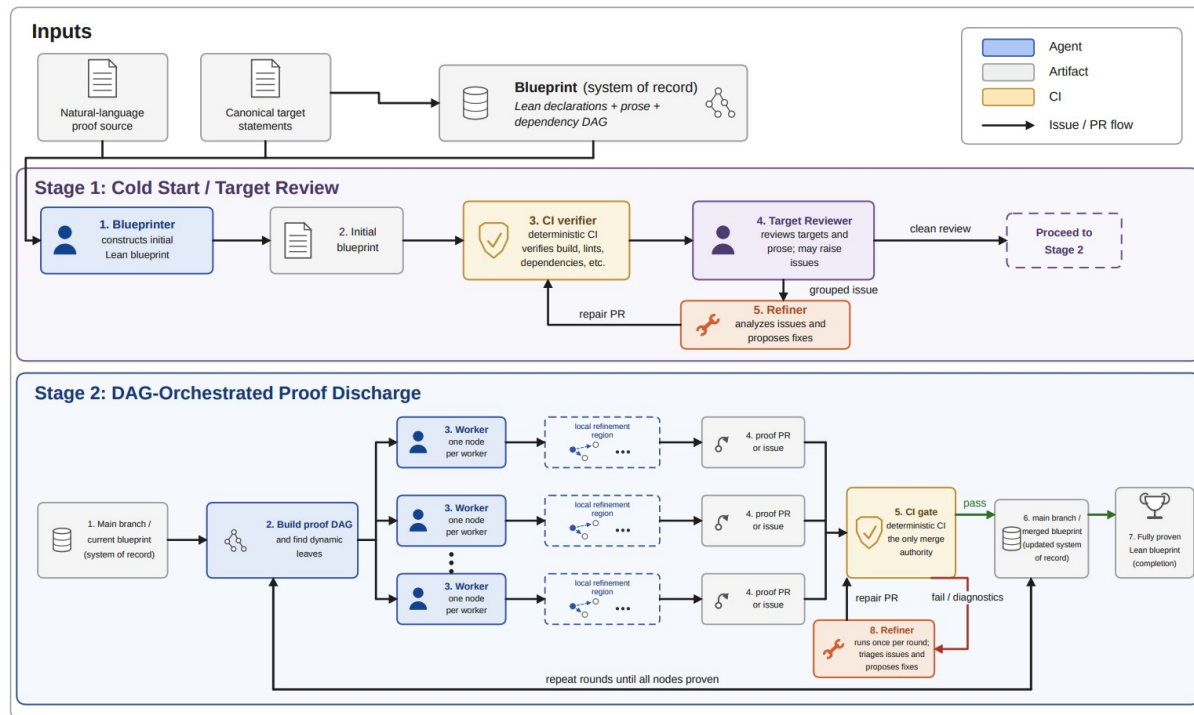
So the question for any paper is not "can Lean express this" — it is how much of what the paper stands on already exists, and how much somebody has to build first.

For a physics paper today, the honest answer is: a great deal has to be built first.

Autoformalization

LeanMarathon: Toward Reliable AI Co-Mathematicians through Long-Horizon Lean Autoformalization

Yuanhe Zhang* Yuekai Sun† Taiji Suzuki‡ Jason D. Lee§ Fanghui Liu¶



FormalScience: Scalable Human-in-the-Loop Autoformalisation of Science with Agentic Code Generation in Lean

Jordan Meadows¹ Lan Zhang² André Freitas^{2,3,4}

¹Independent Researcher

²University of Manchester, UK

³Idiap Research Institute, Switzerland

⁴National Biomarker Centre, CRUK-MI, UK

Correspondence: j.c.meadows@hotmail.com

Key Inputs

Informal proofs $\mathcal{D}$
Gold-standard $\mathcal{D}^*$

1 Informal Statement Generation & Proof Enhancement

$$X = \sum_{d \in \mathcal{D}} S(\mathcal{M}(T_{fs}(d, \mathcal{D}^*) P_a))$$

In-context learning

2 Formal Proof Generation & Iterative Error Correction

$$C^{(t+1)} = \begin{cases} C^{(t)}, & \text{if } \mathcal{L}(C^{(t)}) = (0, \varepsilon) \\ \mathcal{M}'(T_c(x, C^{(t)}, e)), & \text{if } \mathcal{L}(C^{(t)}) = (1, e) \end{cases}$$

Formal classifier

3 Iterative LLM-based Formal Proof Alignment with Expert Classification

$$C^{(k+1)} = \begin{cases} C^{(k)}, & \text{if } \mathcal{H}^{(k)} = 0 \\ \mathcal{R}(\mathcal{M}'(\mathcal{M}'(T_g(x), C^{(k)}), P_g)), & \text{otherwise} \end{cases}$$

Expert classifier

4 Post-processing & Verification

Re-application of 2 → 3 after proof separation & individual proof verification via formal system

Figure 1: An overview of the FormalScience approach.

a paper

frozen.
every byte fixed,
so pointers hold.

a paper

frozen.
every byte fixed,
so pointers hold.



extract the claims

- what it says
- where it came from
- a fingerprint

a paper

frozen.
every byte fixed,
so pointers hold.

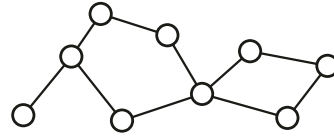


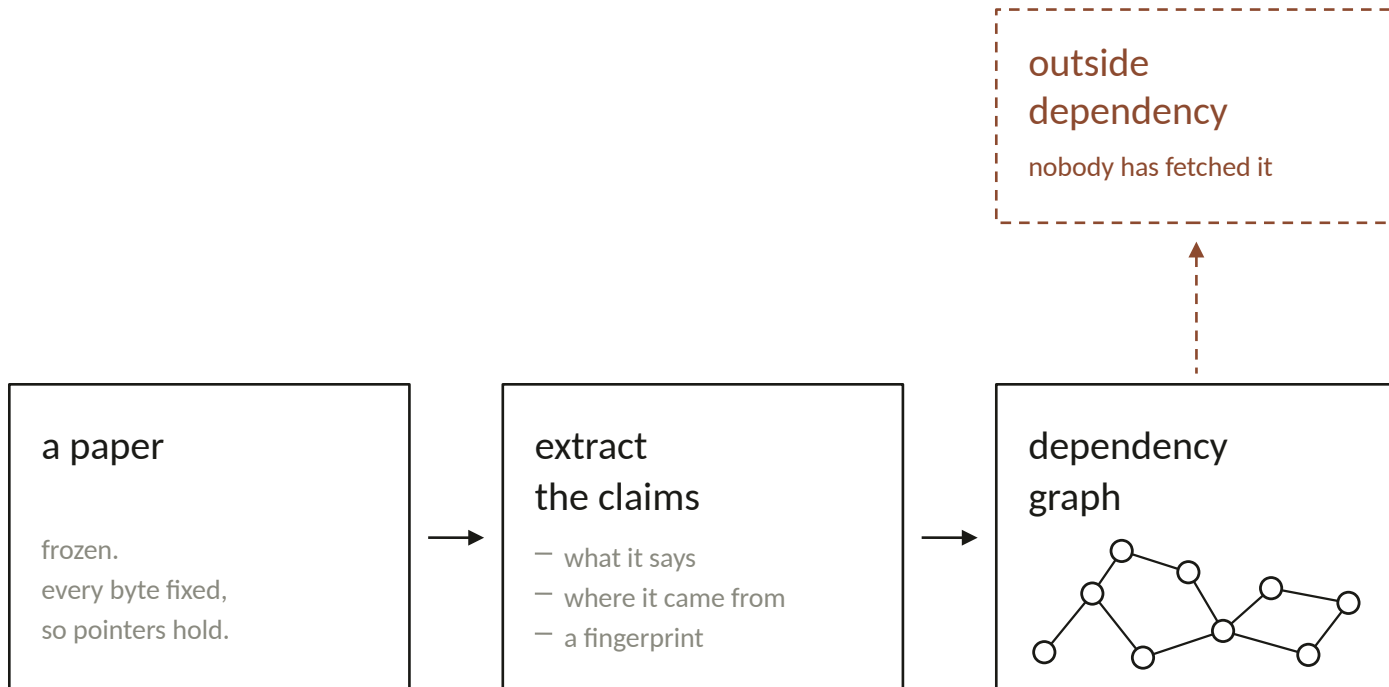
extract the claims

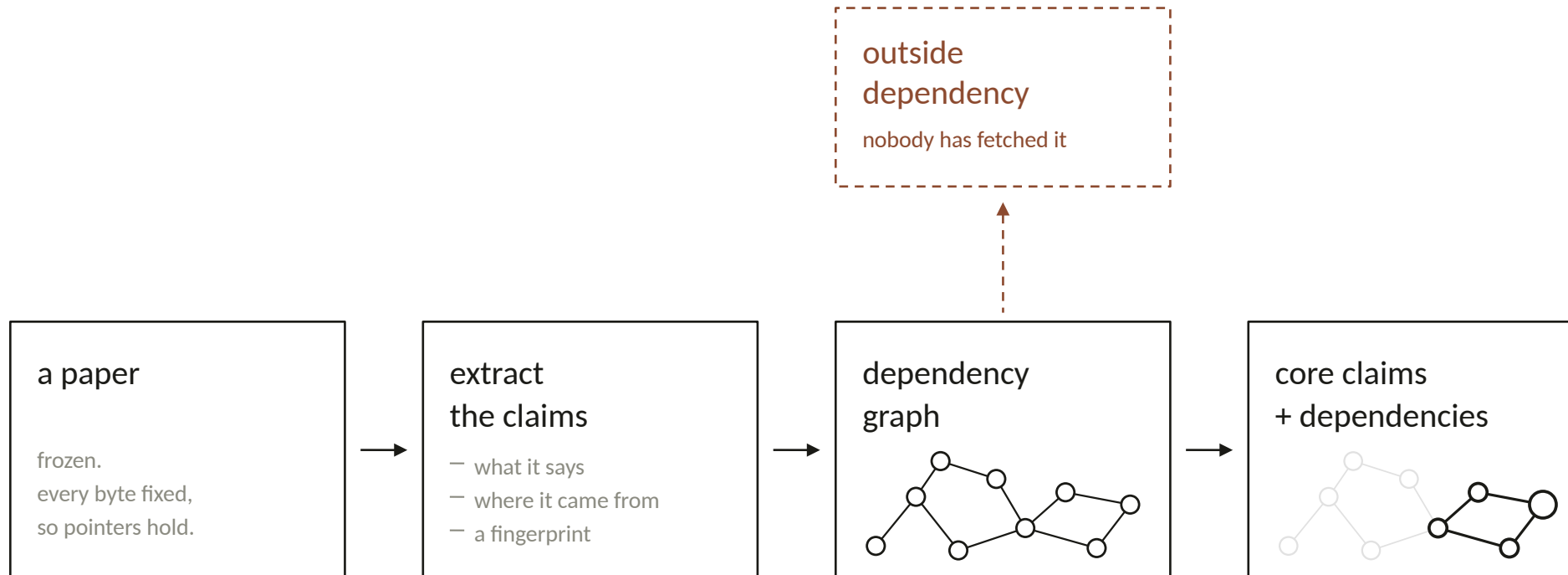
- what it says
- where it came from
- a fingerprint

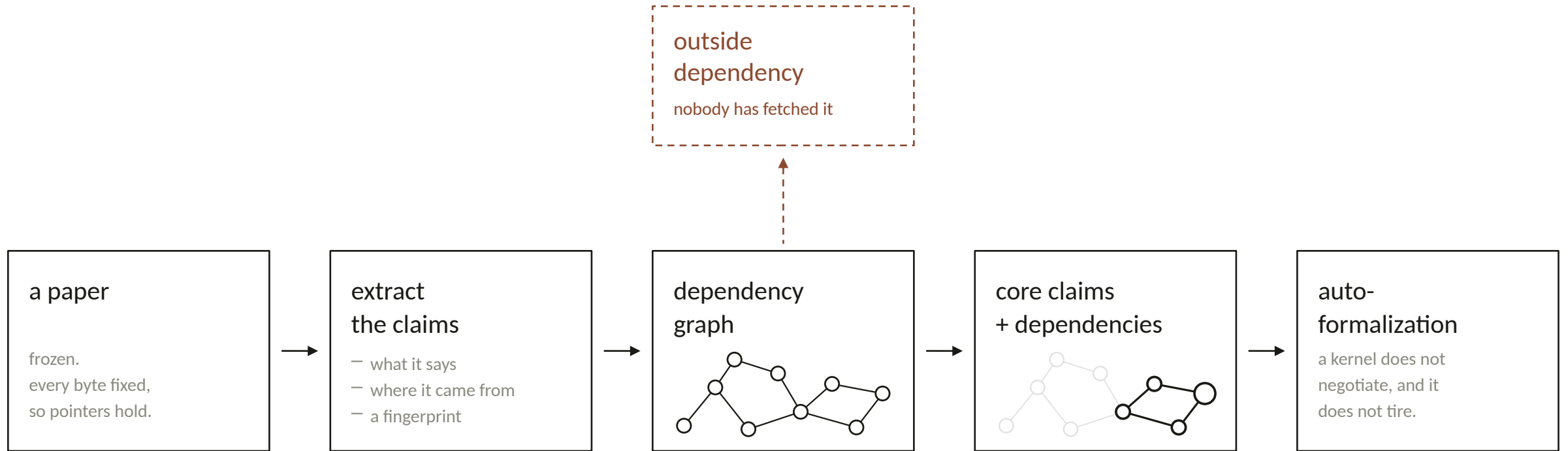


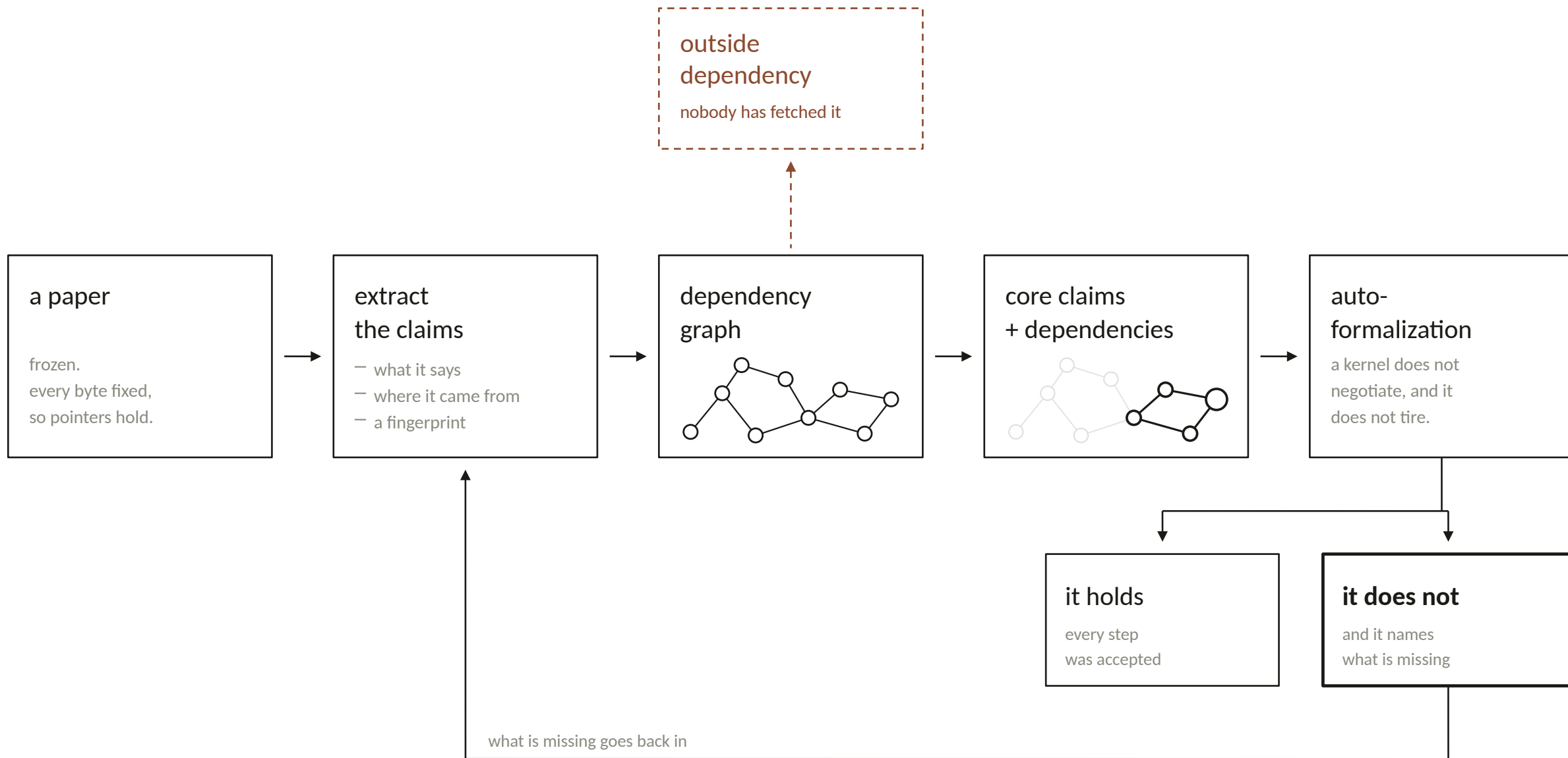
dependency graph











math-claim-ir · the theorem this chain ends at

claim.id	closed-form-equality
statement_kind	theorem
source.occurrence_work	arxiv:2607.26154v1

as the paper states it

$$\text{RoM}_M(\rho) = \max(1, \max_Q \sum_{P \text{ in } Q} |\text{tr}(P \rho)|)$$

structured_statement — what that one sentence actually contains

assumptions.explicit	M has no active dependencies G_M is perfect
assumptions.source_implicit	m >= 1 found in the PROOF, not the statement
quantifiers	4 — 3 SOURCE_IMPLICIT (n, m, M), 1 SOURCE_EXPLICIT (rho)
local_binders	Q over Cliques(G_M)
conventions	The empty clique is allowed and contributes 0.
conclusion	$\text{RoM}_M(\rho) = \max(1, \max_{Q \text{ in } \text{Cliques}(G_M)} \sum_{P \text{ in } Q} \text{tr}(P \rho))$

normalization.relation_to_source	LOGICAL_FORM_EXPANDED
normalization.unresolved_symbols	[] nothing left unresolved
claim.content_hash	sha256:9168a6687534c0387f16dec4e334d18b...
semantic_content_hash	sha256:bfd91f0b1a13fec71d05f5e4e5f0e890c...
artifact.artifact_hash	sha256:46b60671563656ba0e1e919f833599a84...

one sentence in the paper. four quantifiers, three assumptions and a convention in the record.

lamport.json · the structured proof the model returned

1. Fix a finite index type ι and coefficients $x : \iota \rightarrow \mathbb{R}$ satisfying $\sum i, x\ i = 1$.
2. Apply `AgtXlv.RoM.normalized_l1` to x and the normalization hypothesis; project its first conjunct to obtain $1 \leq \text{AgtXlv.RoM.l1Cost } x$.
3. Unfold `l1Cost` to express the bound as $1 \leq \sum i, |x\ i|$.

alignment_notes

- `Fintype` represents the finite index type; x has real coefficients and the hypothesis preserves their sum being one. No nonnegativity or nonempty-index assumption is added.
- The explicit sum of absolute values represents the source's coefficient cost. Only the lower-bound conjunct of `normalized_l1` is used.
- This statement concerns the coefficient bound alone; it does not assert primal feasibility or strong duality.

remaining_obligations

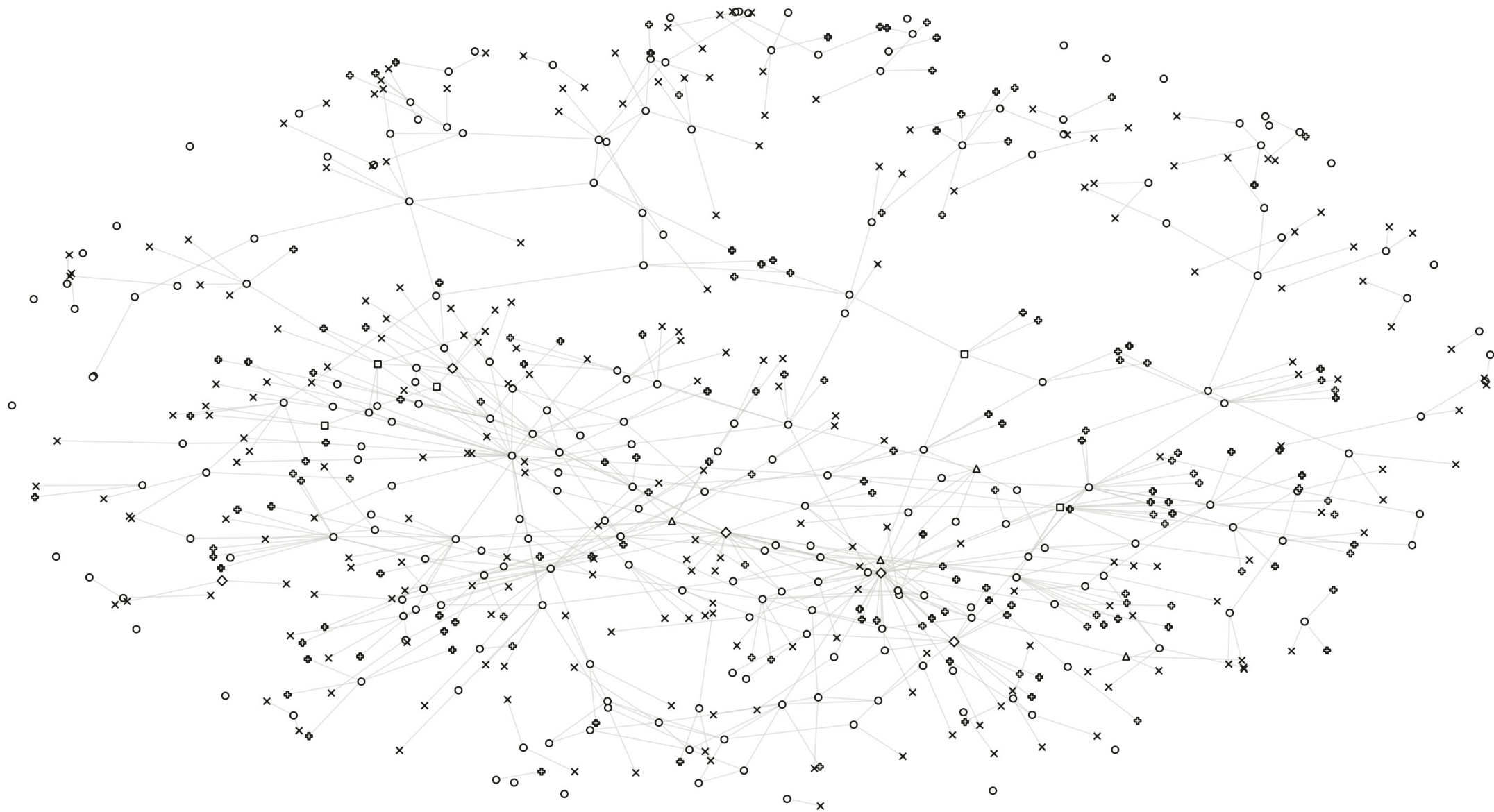
- **Host elaboration of the proposed terms remains pending; no Lean execution was performed.**

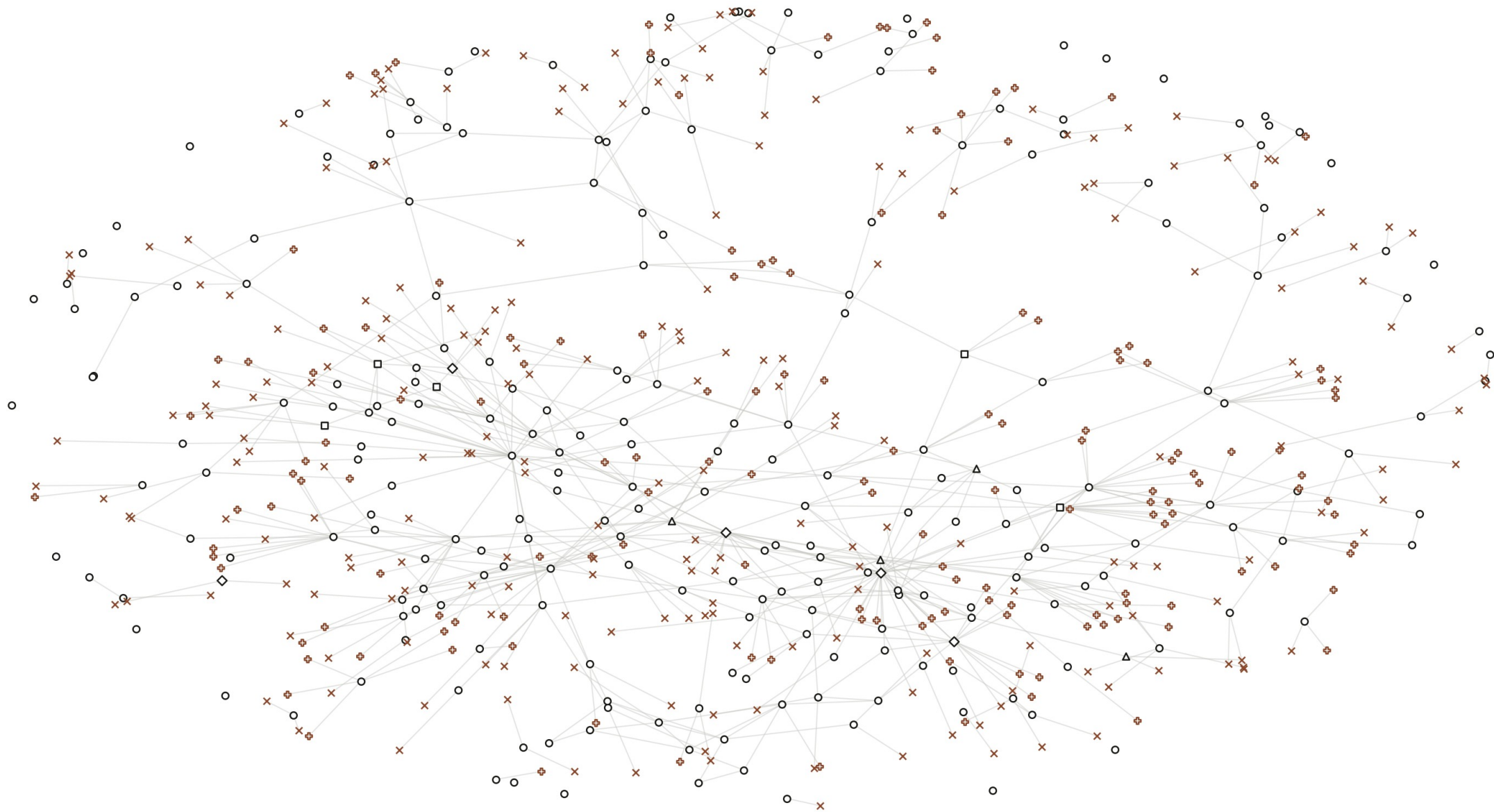
ledger.json · and what the host did next

<code>result.kernel_checked</code>	<code>true</code>
<code>audit_record.declaration</code>	<code>AgtXlv.Generated.Proof_a29f133fb6f82da97507</code>
<code>audit_record.axioms</code>	<code>propext, Classical.choice, Quot.sound</code>
<code>result.lean_source</code>	<code>488 bytes</code>

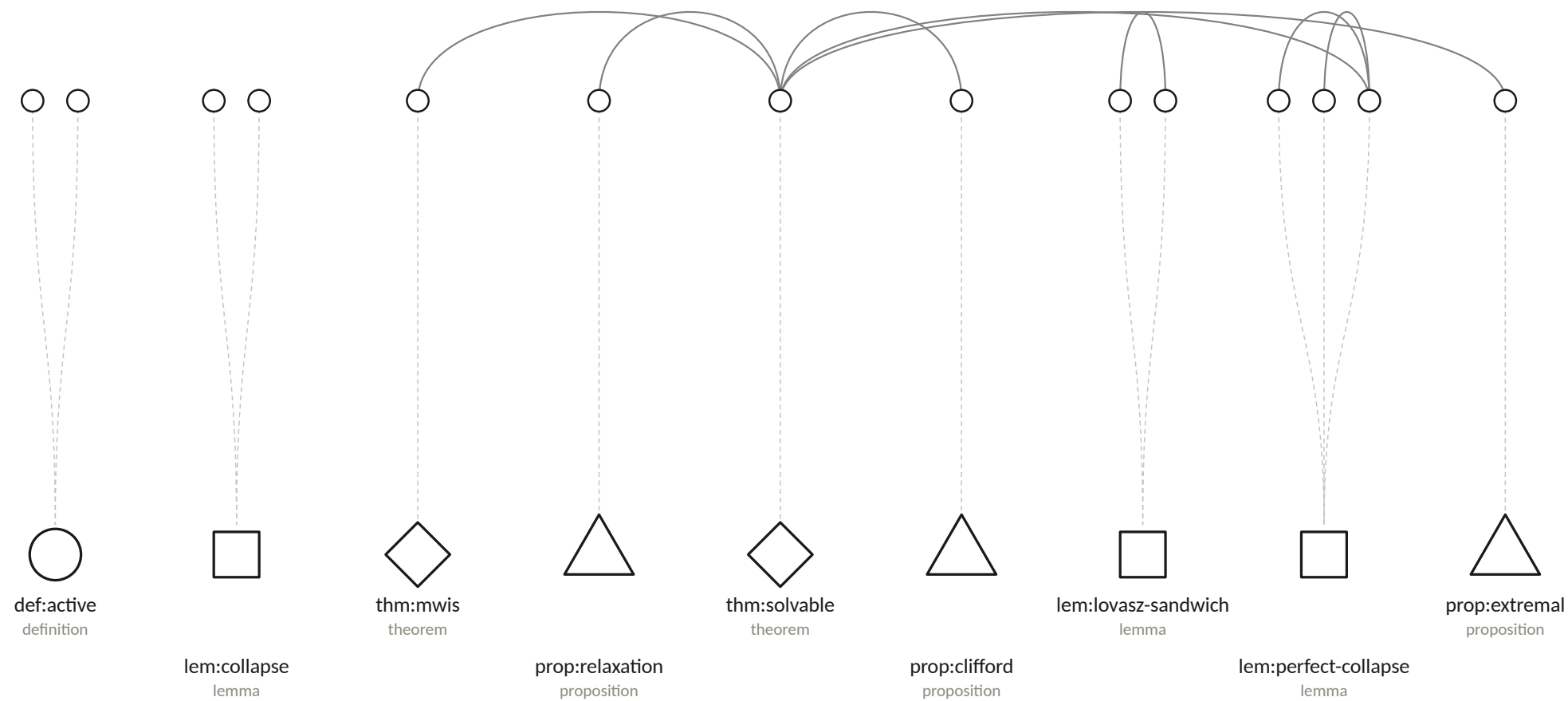
the obligation above was outstanding when the model wrote it.

the host then elaborated the terms and the kernel accepted them.





14 candidates that land inside a declared result (one mark each, kind not shown)



the 9 results the paper actually declares





COLOUR = SOURCE

○ this paper

● Chvátal 1970s

● Gottesman 1997

● Veitch 2013

● Howard 2016

● Handbook 2024

● Varela 2026

the theorem



L8

L7

L6

L5

L4

L3

L2

L1

L0

COLOUR = SOURCE

○ this paper

● Chvátal 1970s

● Gottesman 1997

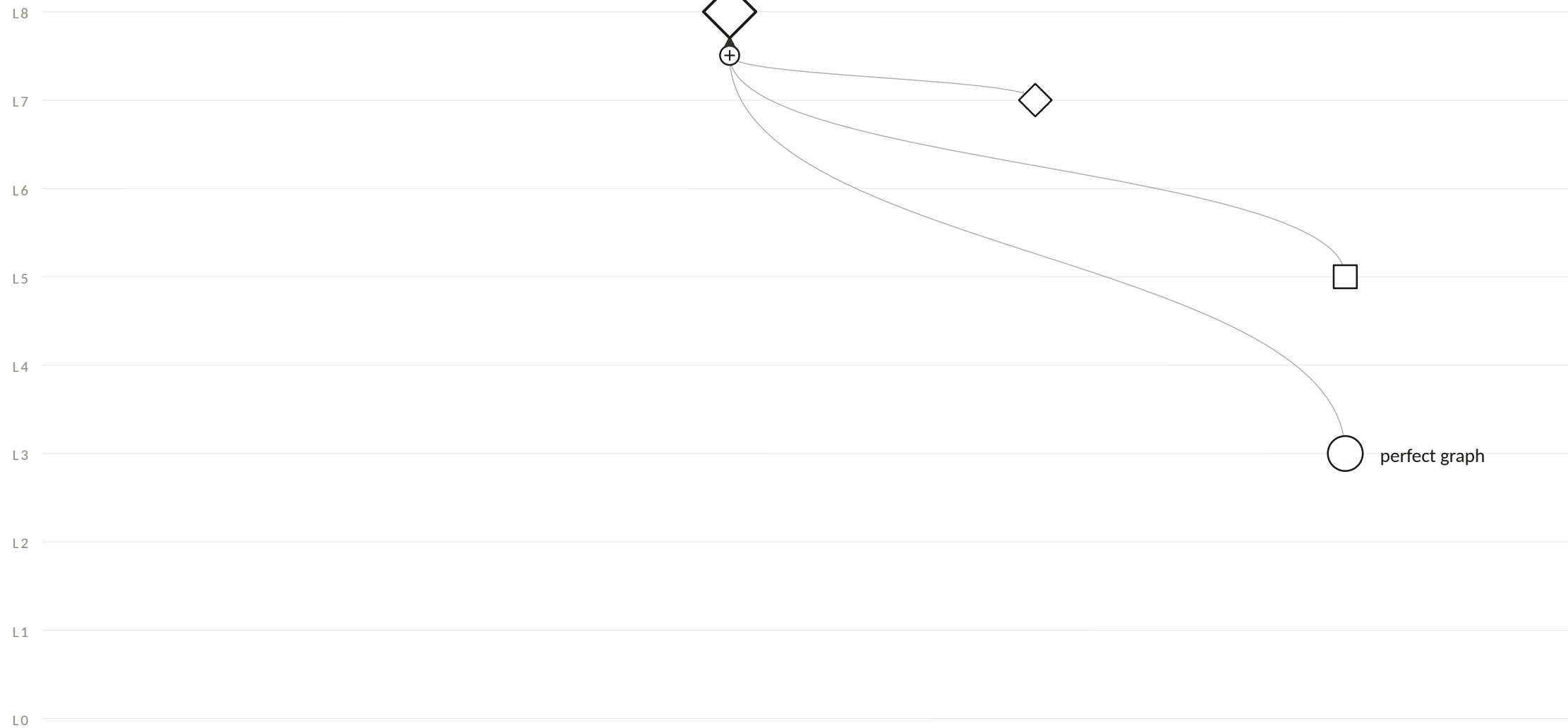
● Veitch 2013

● Howard 2016

● Handbook 2024

● Varela 2026

the theorem



the theorem

COLOUR = SOURCE

○ this paper

● Chvátal 1970s

● Gottesman 1997

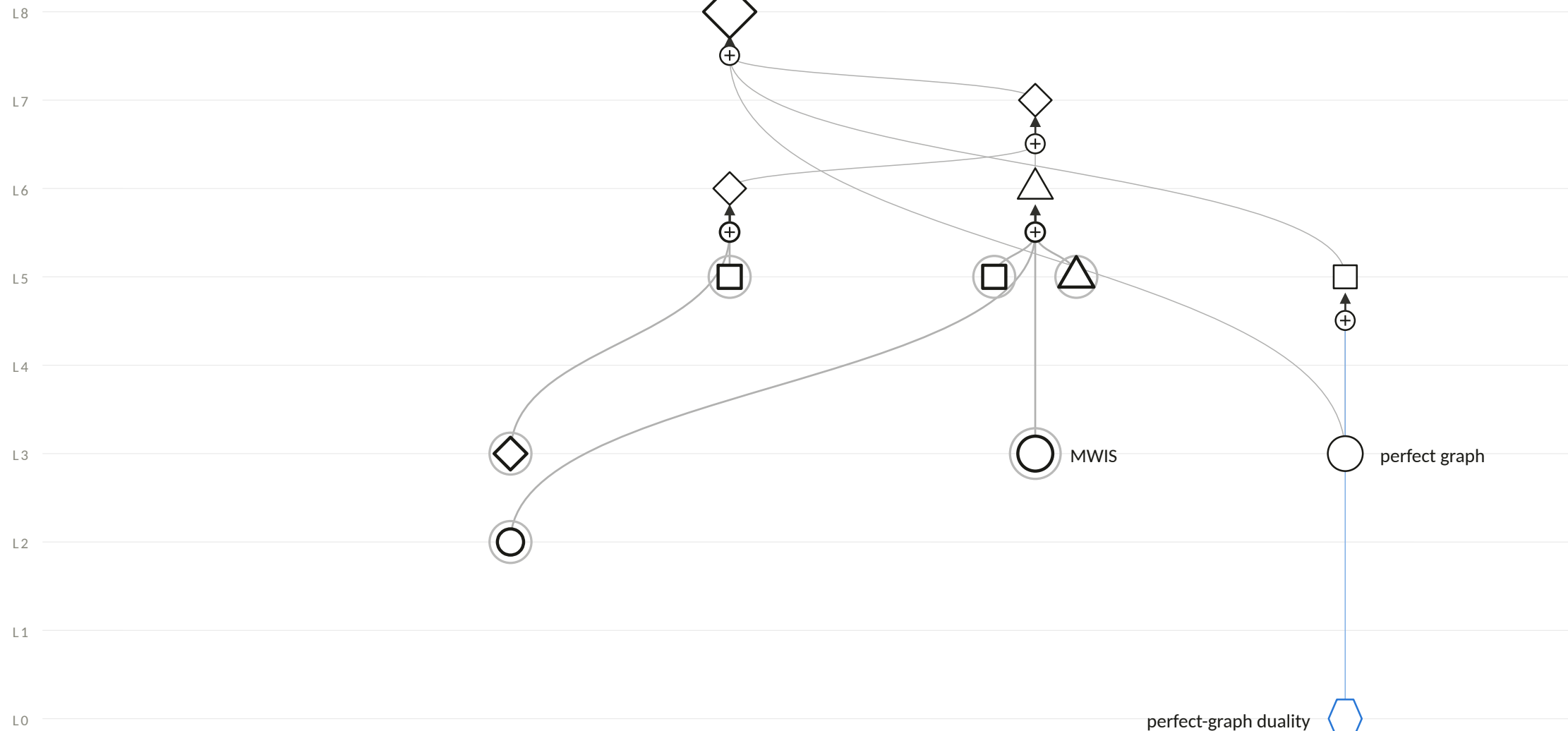
● Veitch 2013

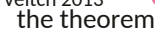
● Howard 2016

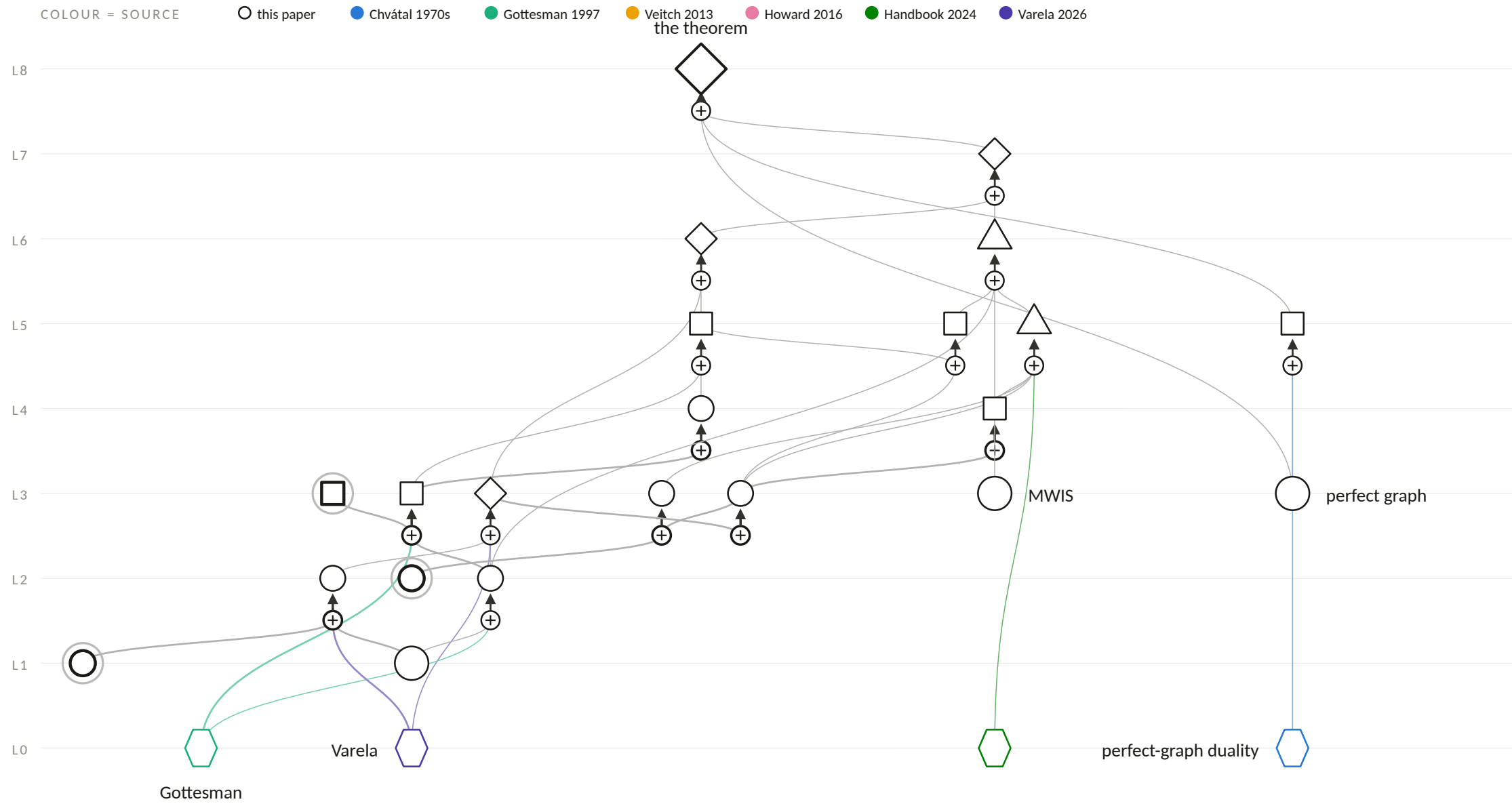
● Handbook 2024

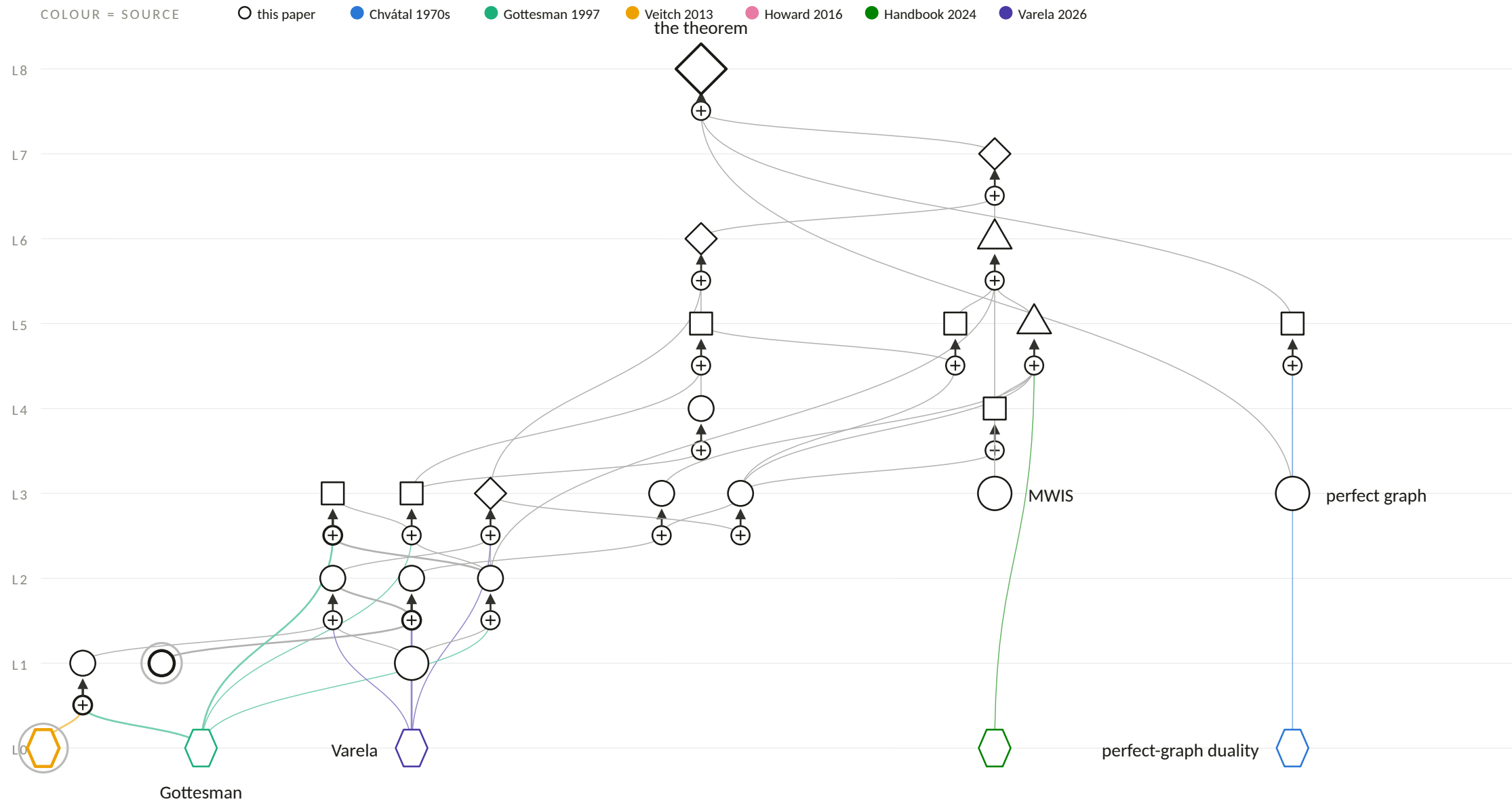
● Varela 2026

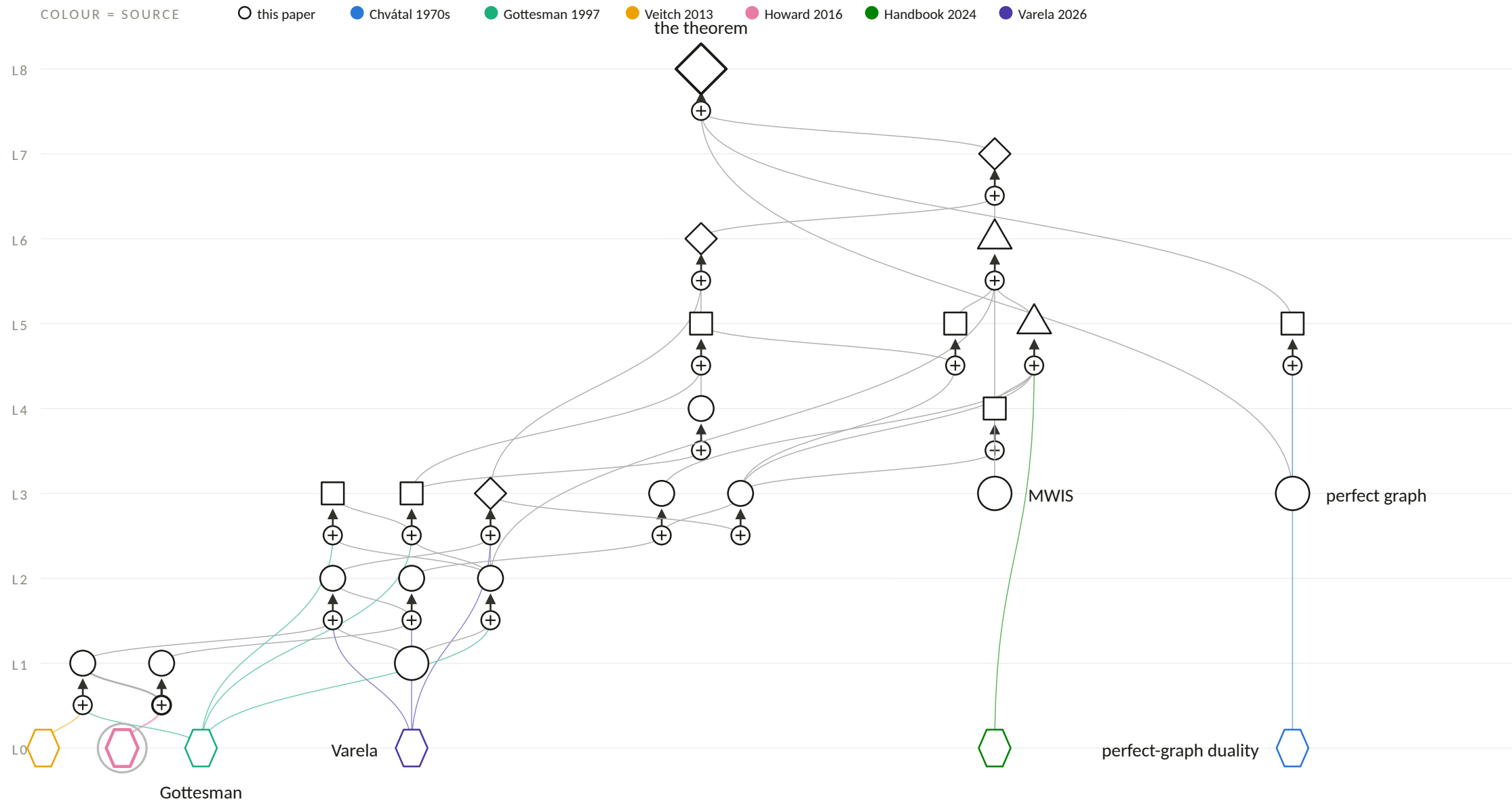
the theorem

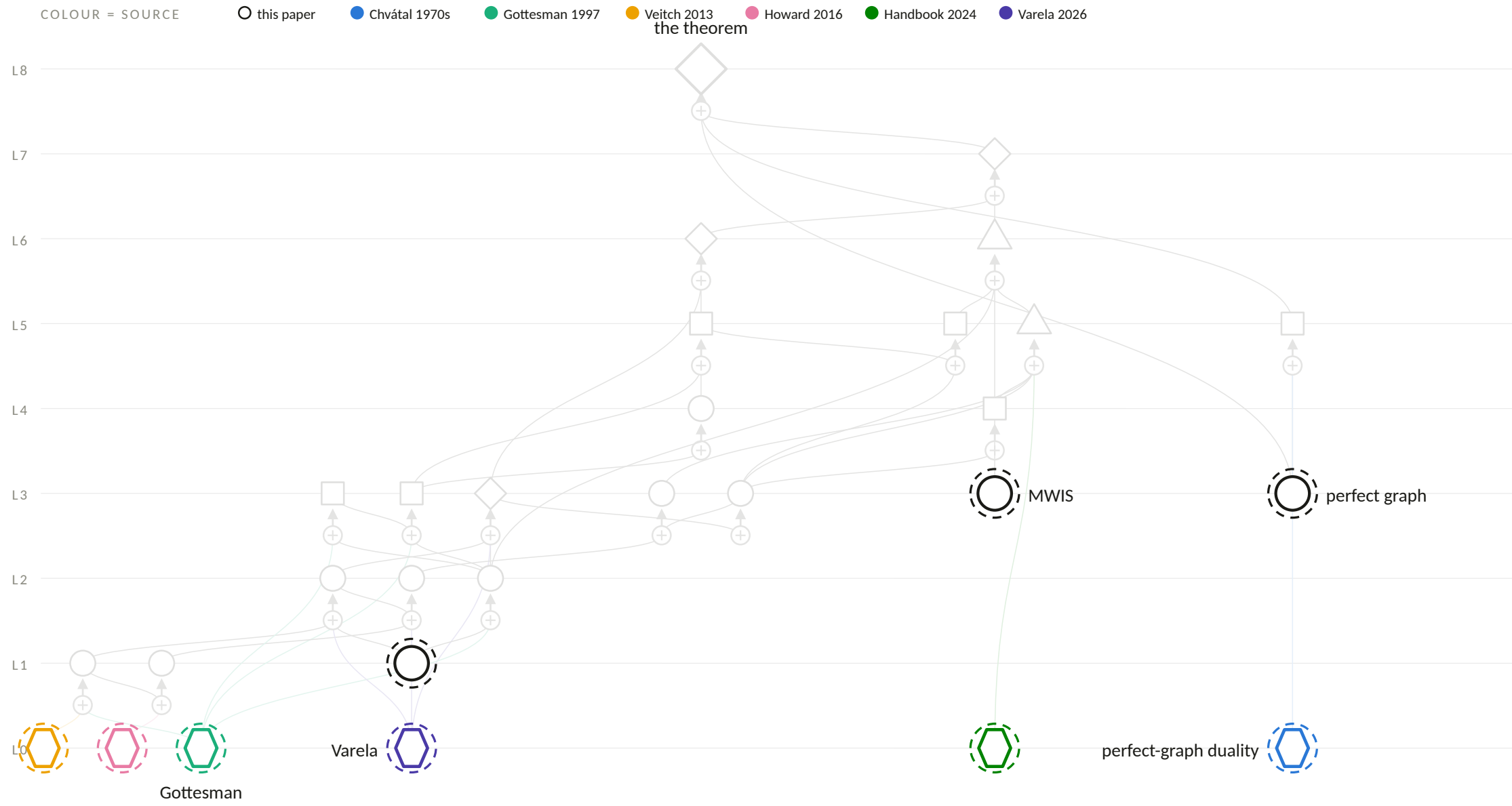


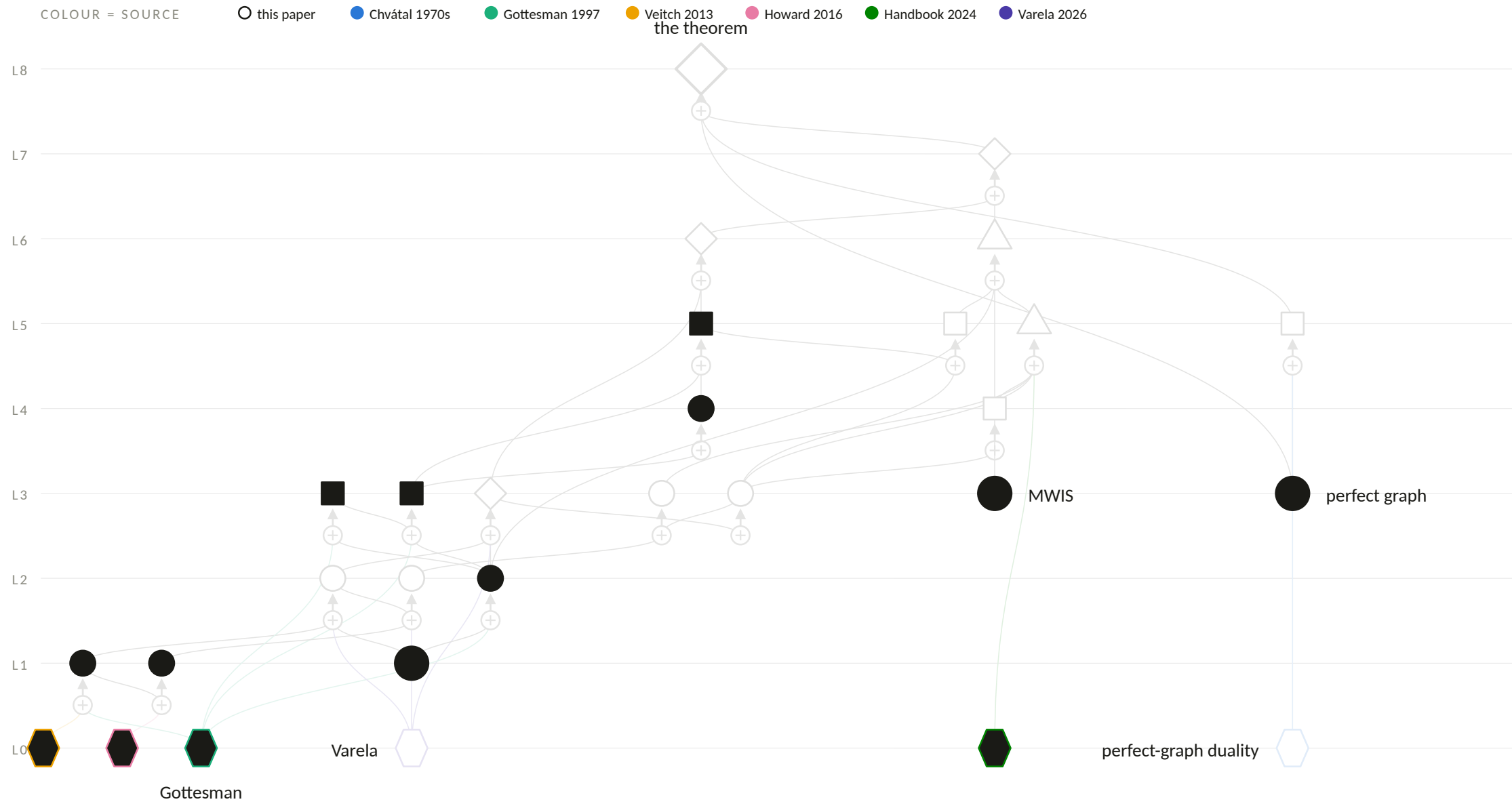


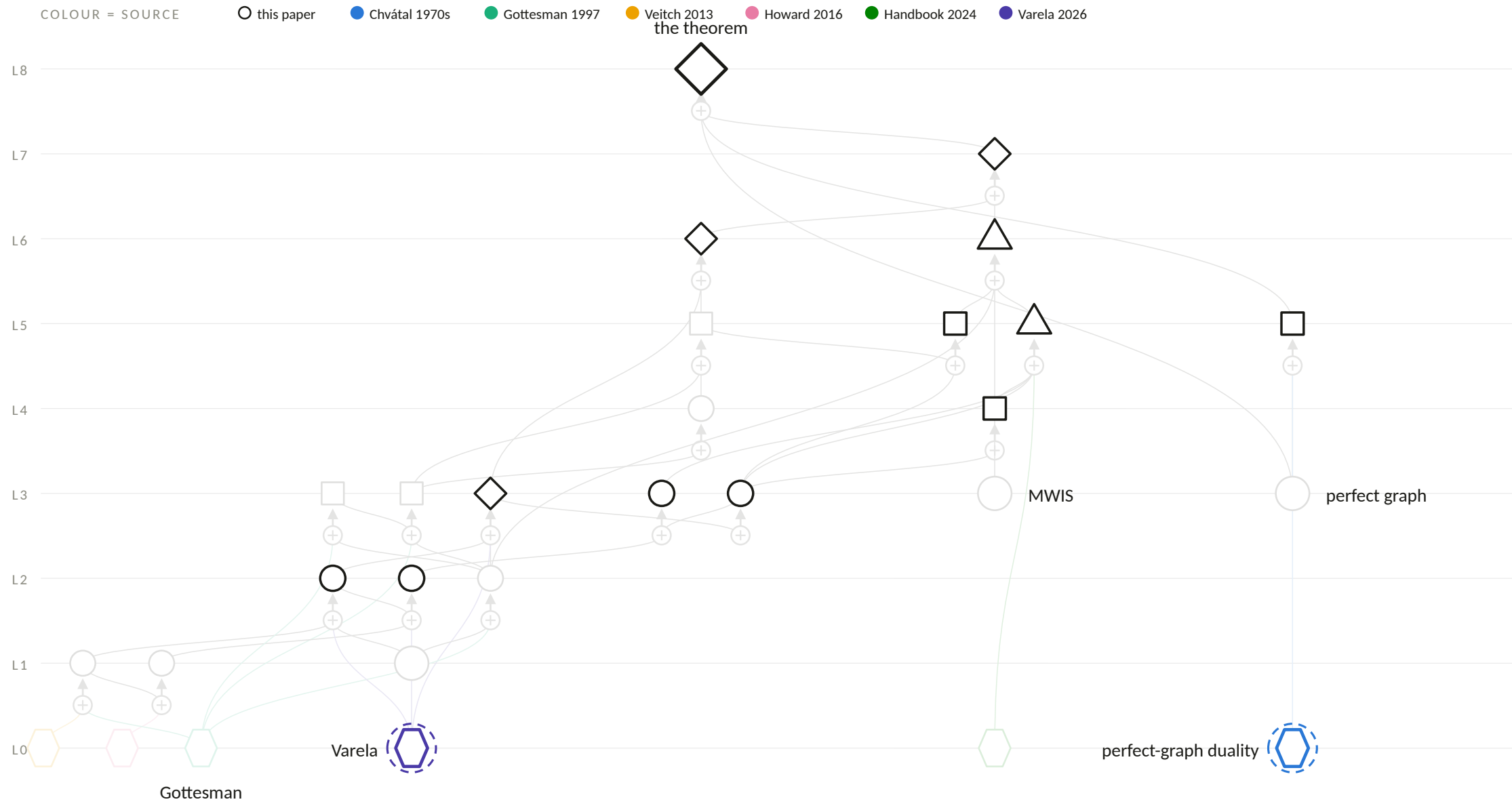






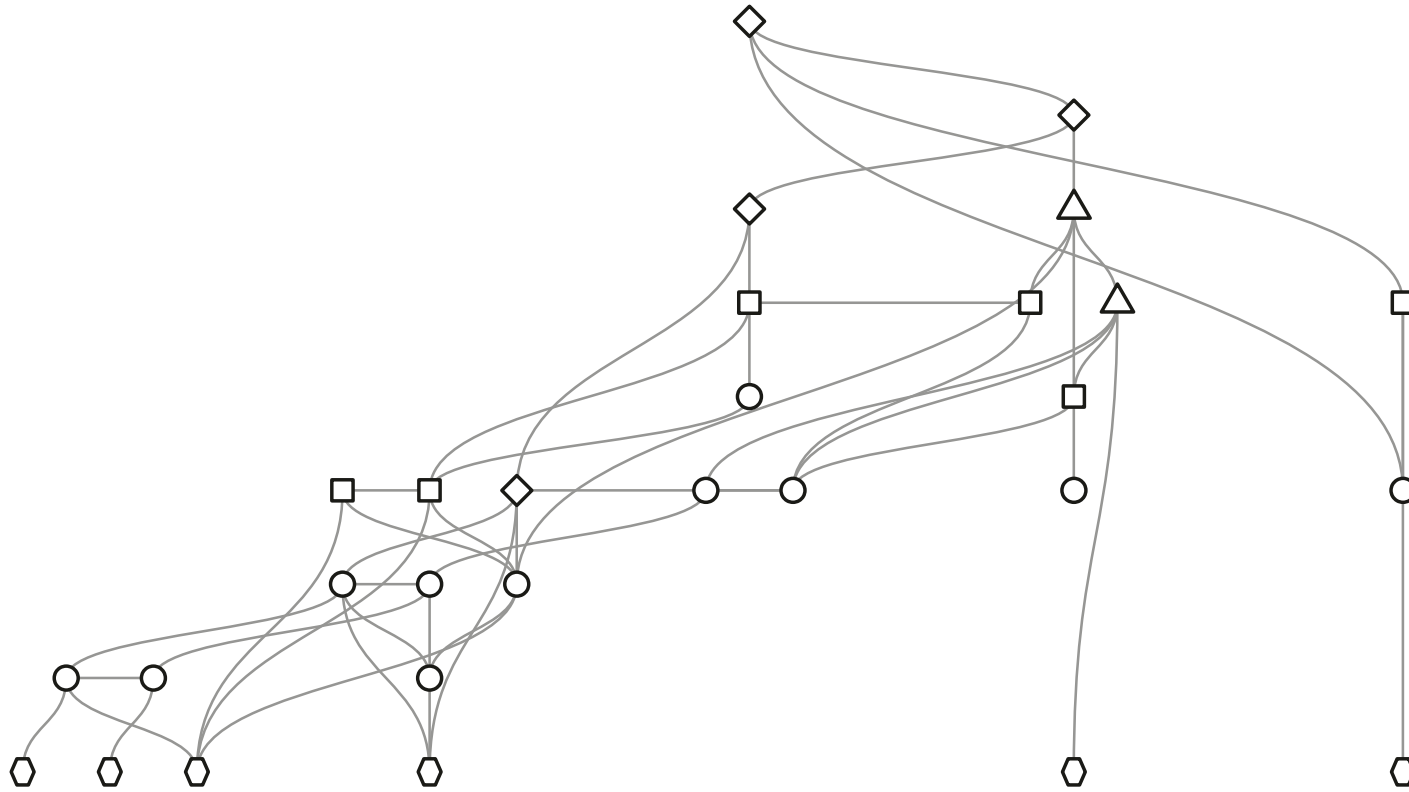






One theorem, and everything it rests on

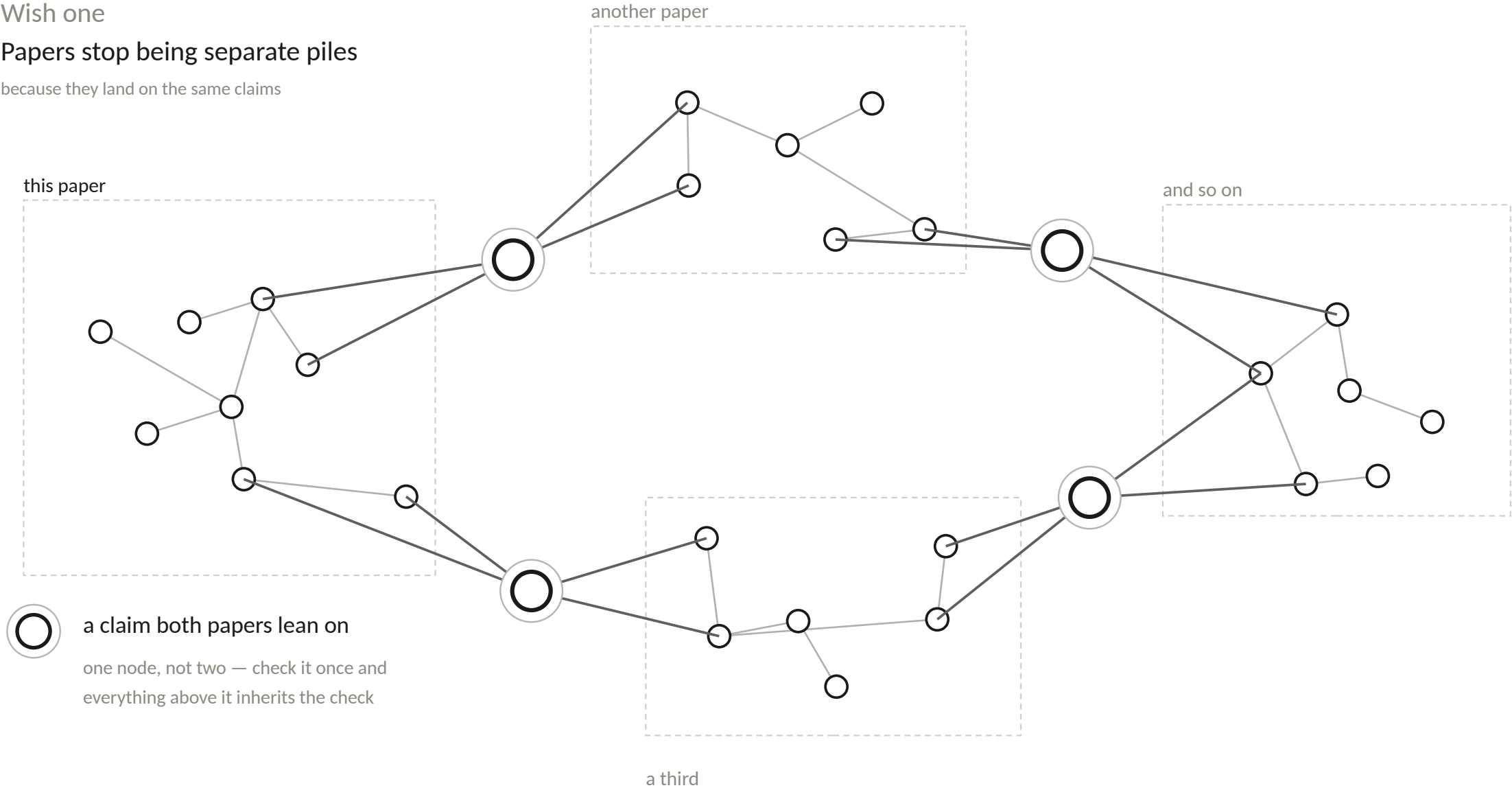
29 claims and 46 dependencies — the object the rest of this act works on



Wish one

Papers stop being separate piles

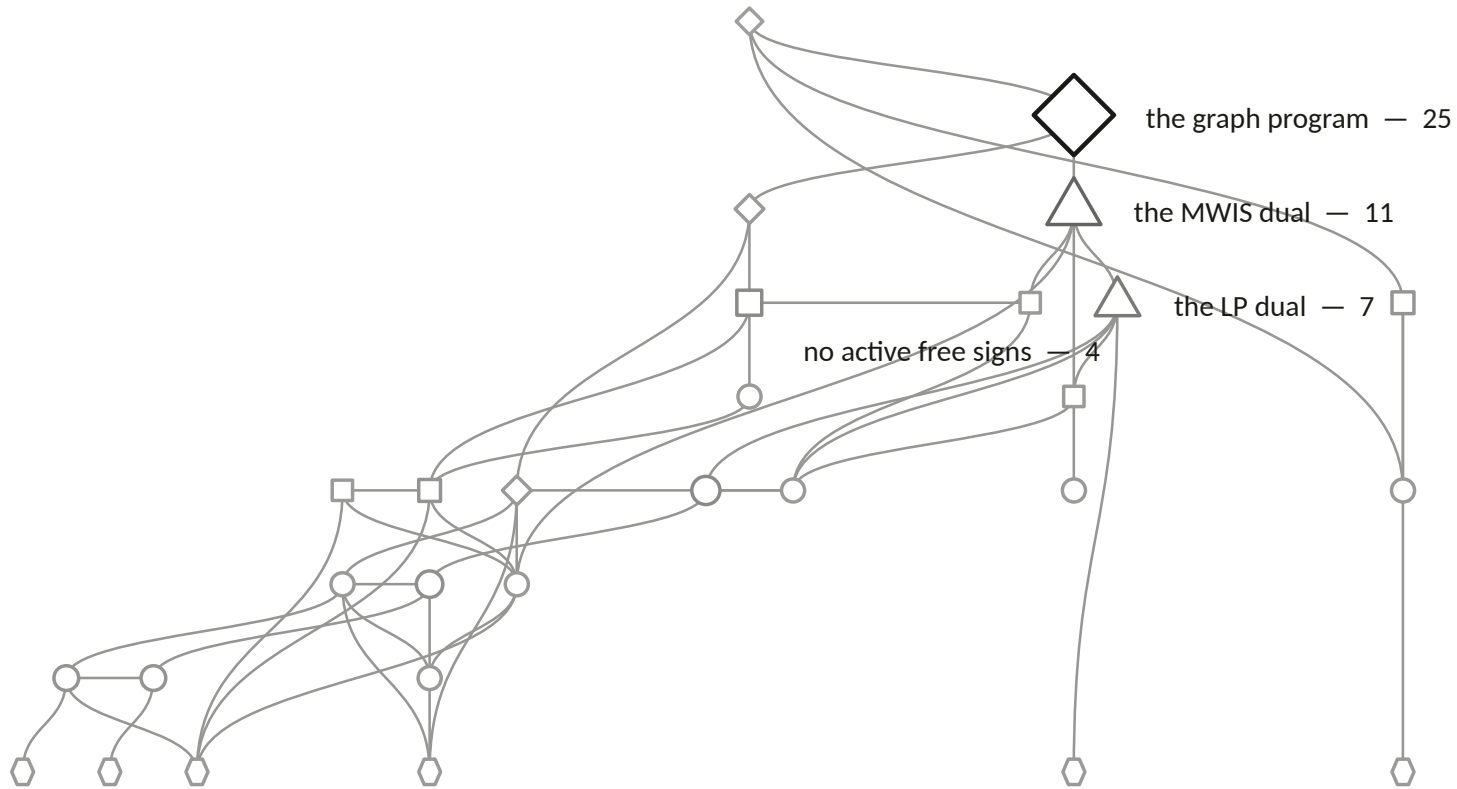
because they land on the same claims



schematic

Which claims actually carry the load

remove one claim, recompute the closure, count what leaves — size is what is lost

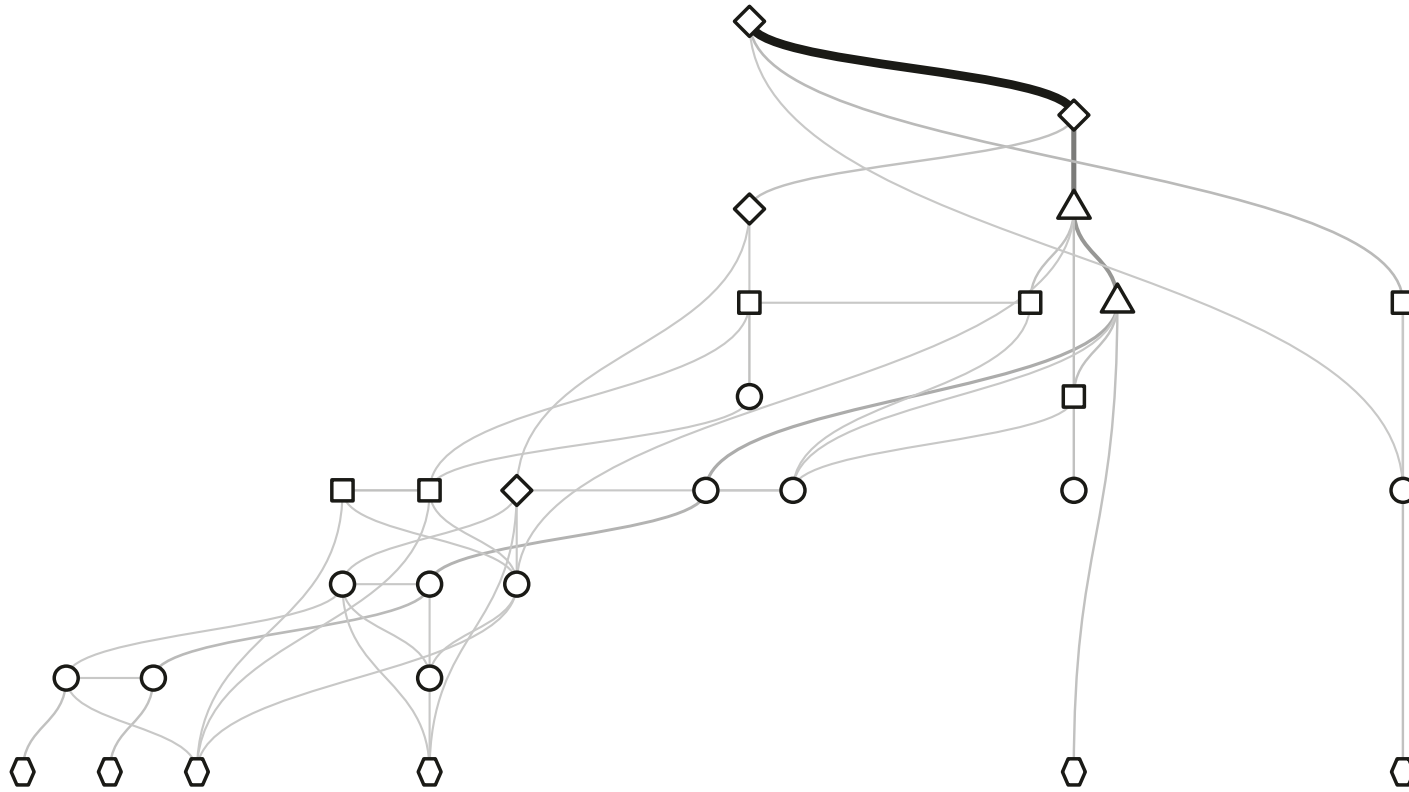


claims lost if this one is removed



The same question, asked of every dependency

46 links, each weighted by what removing it costs

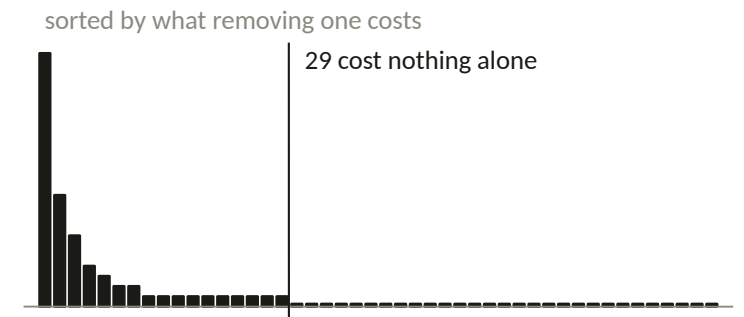
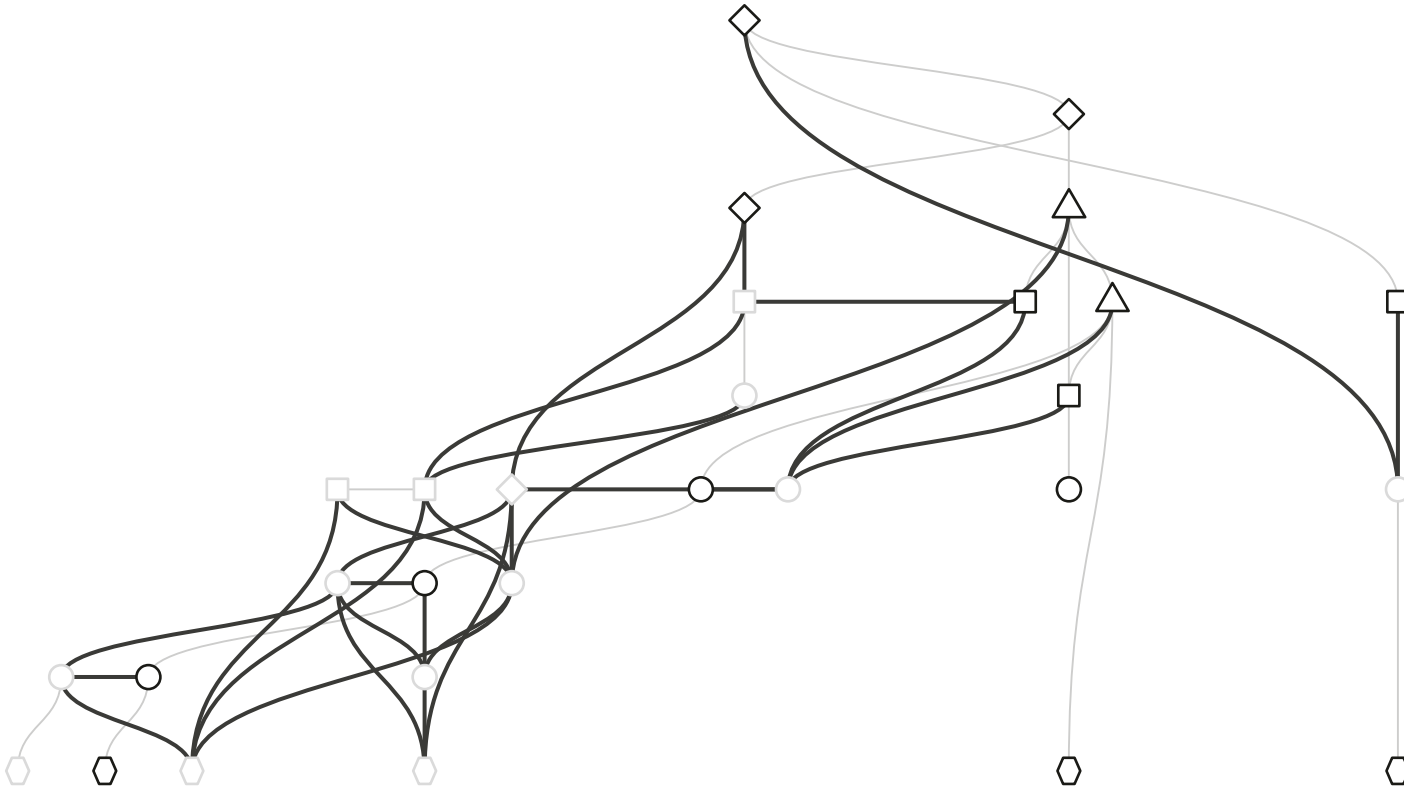


sorted by what removing one costs



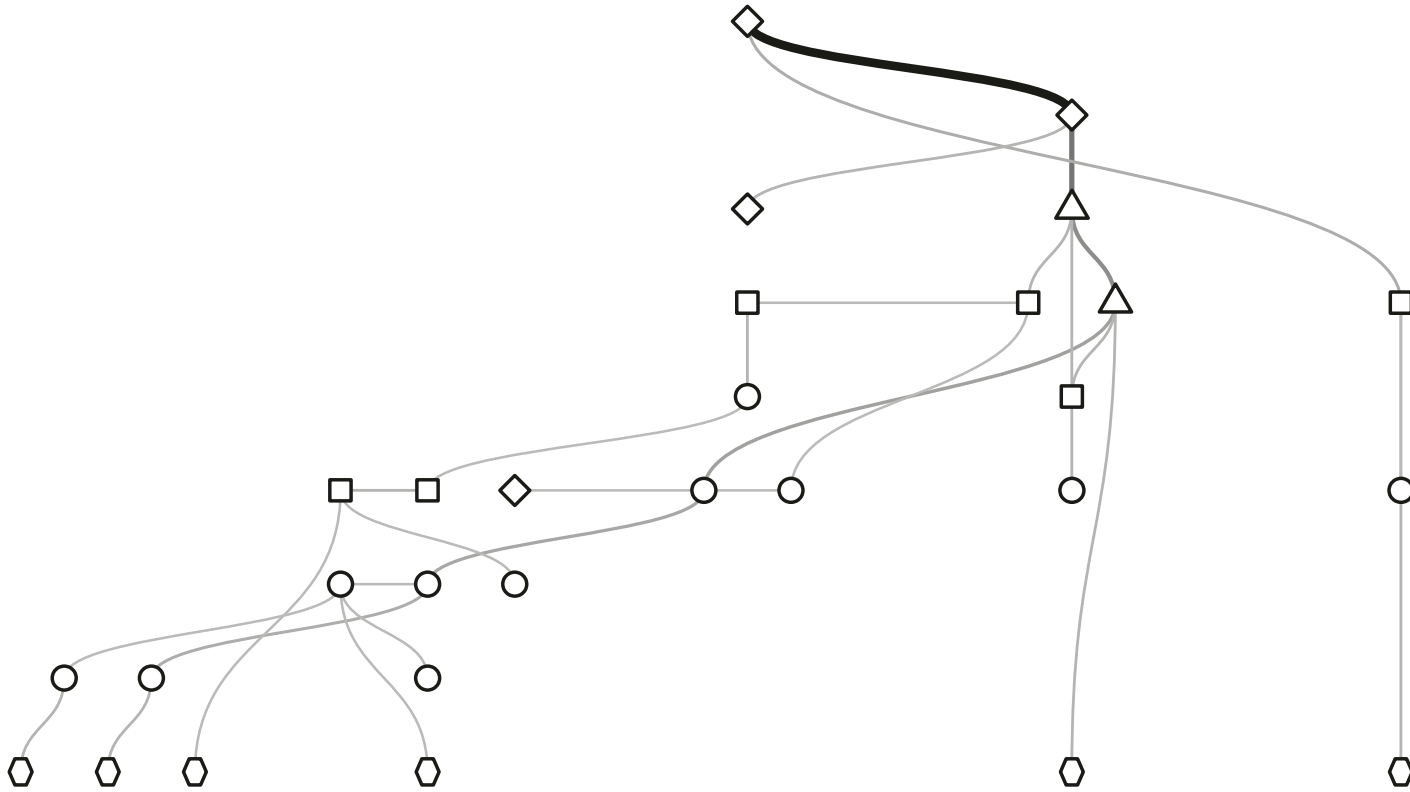
Alone every one of these 29 is free

take them away together and 14 of the 29 claims leave the closure



The skeleton

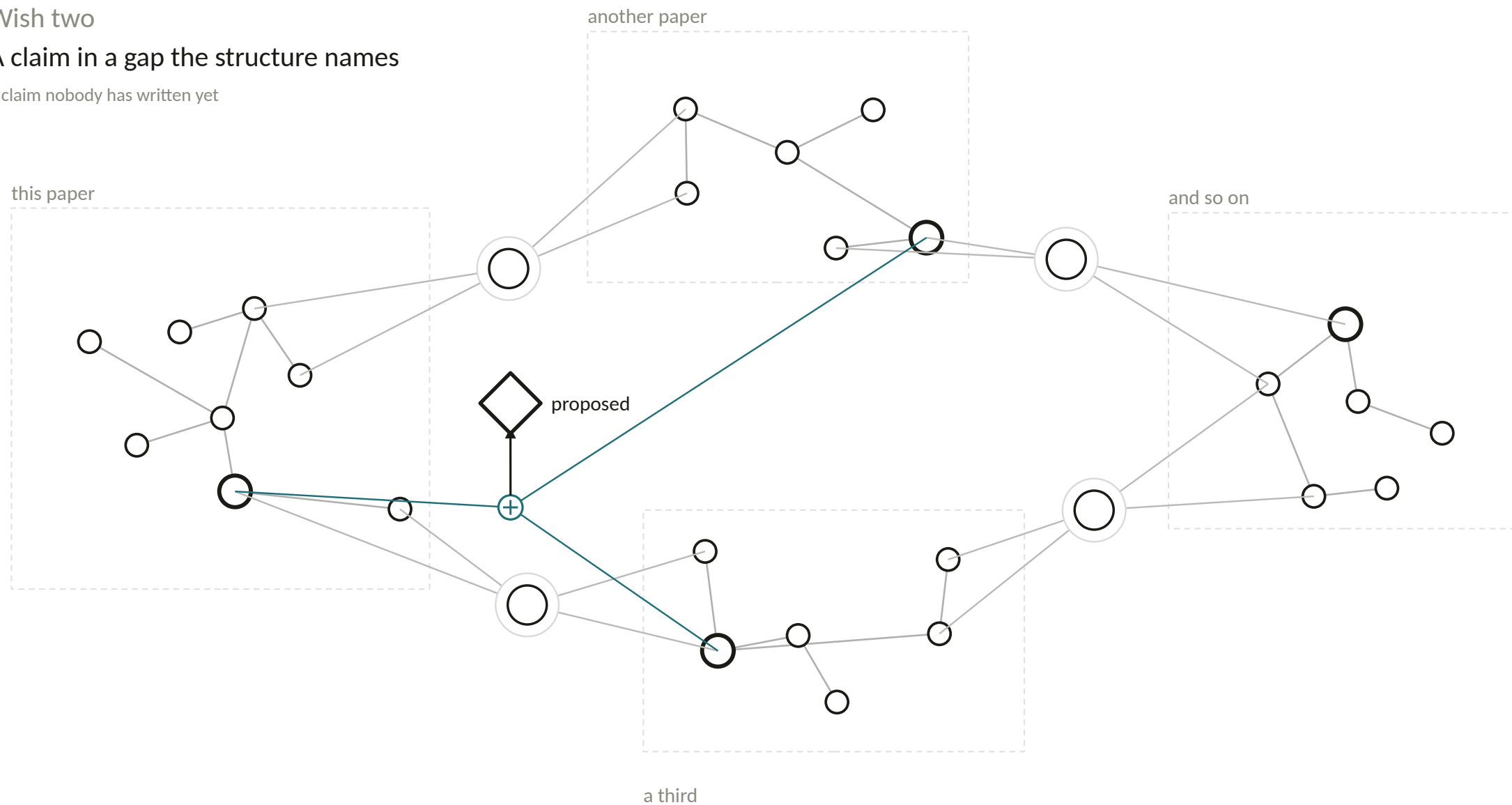
28 dependencies, and every claim still reachable



Wish two

A claim in a gap the structure names

a claim nobody has written yet



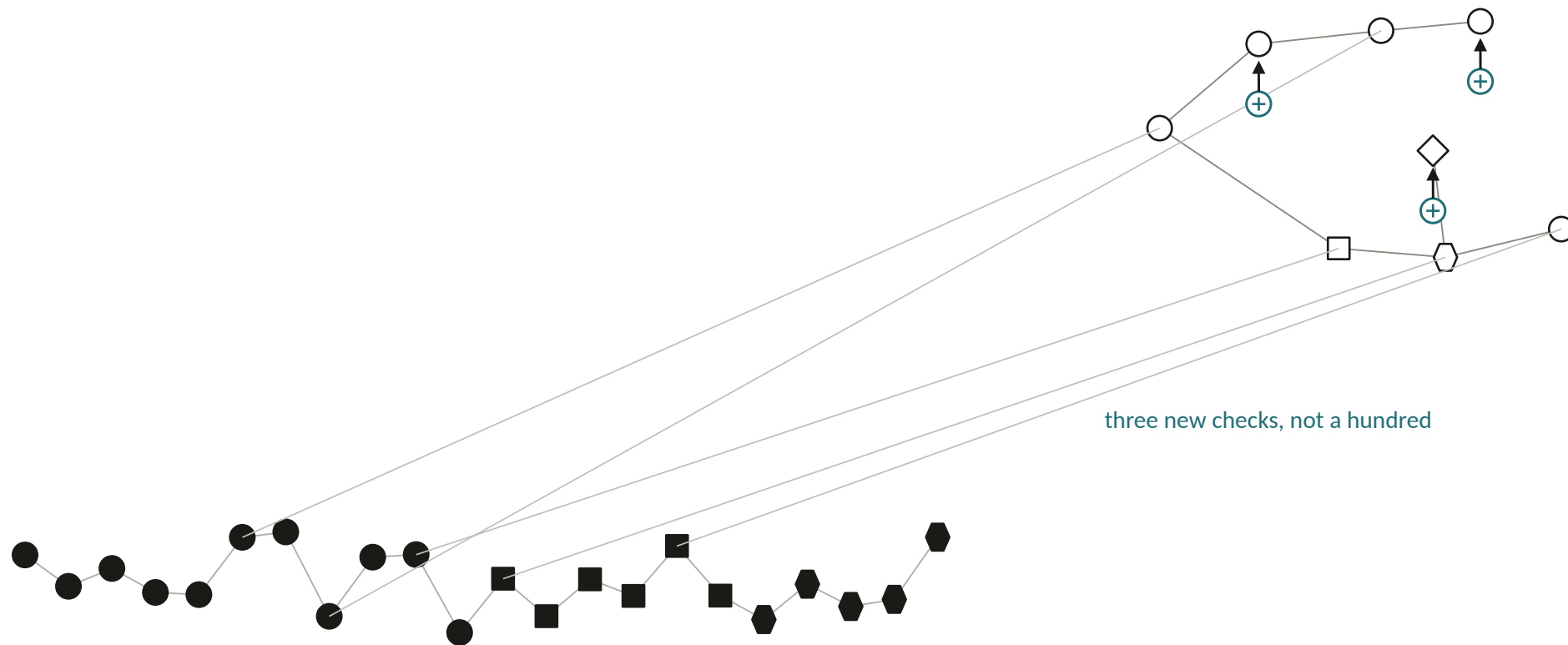
schematic

Wish three

A new paper inherits what is checked

and only three of its claims are genuinely new

a new paper



three new checks, not a hundred

already checked — and it stays checked

Where this actually is

shown

One paper, frozen. 29 claims and 46 dependencies for one theorem, and a chain that runs.

shown

Removing a claim has a measurable cost, so the structure can say which parts carry load.

not yet

The 613-claim extraction is not usable: every node is blocked pending review, and nothing is accepted.

not yet

The chain rests on a DAG written by hand, for this paper. Automatic library lookup is zero lines of code.

not shown

Whether the formal object means the physics you had in mind. No kernel judges that.

The claim is not that this works. It is that it is now small enough to argue about.

Everything on this frame is one paper and one run — a possibility with a receipt, not a result.

Three questions I do not have answers to

A golden age for theoretical research. But for theoretical researchers?

- 1 Does more compute and more intelligence excite you — or frustrate you?
- 2 If a machine can produce the result, which part did you want to do?
- 3 What would have to be checked before you would build on it?

yingjian@lorentz.leidenuniv.nl

the deck, the records and the logs are all in the repository

